자연어처리와 트랜스포머

교수자용 강의자료

단원 목표

1

자연어처리의 개념과 발전 과정, 활용 분야를 알아본다

2

순환신경망의 구조와 장기 의존성 문제, 이를 보완한 LSTM·GRU를 이해한다

3

토큰화와 임베딩을 통해 단어를 벡터로 표현하는 방법을 알아본다

4

어텐션의 개념과 트랜스포머의 전체 구조(인코더와 디코더)를 이해한다

5

셀프 어텐션, 멀티 헤드 어텐션, 피드포워드 신경망으로 이루어진 인코더 블록의 동작을 이해한다

6

디코더가 다음 토큰을 예측해 문장을 생성하는 과정과 트랜스포머의 학습·추론, 활용 사례를 알아본다

7.1

자연어처리 기초

7.1.1 자연어처리 개요
7.1.2 순환신경망과 장기 의존성 문제
7.1.3 단어의 벡터화: 토큰과 임베딩

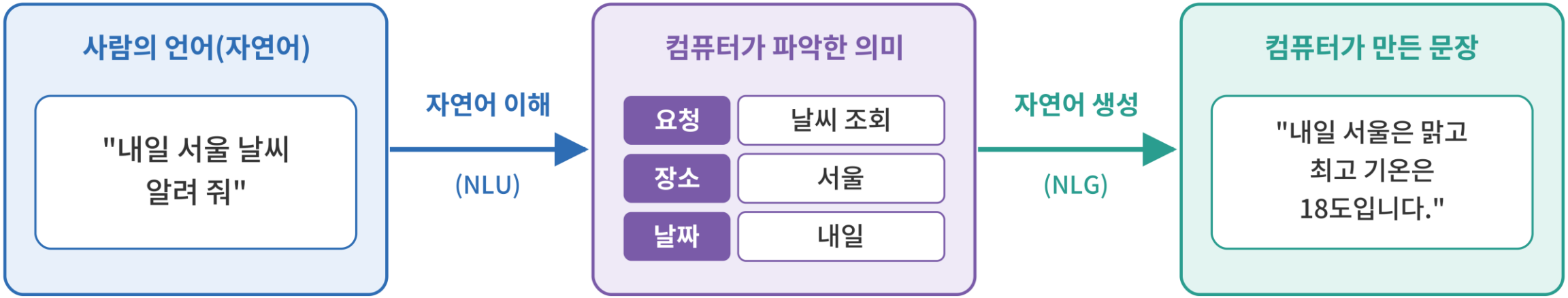
자연어처리의 정의

- 자연어(natural language): 일상에서 쓰는 한국어, 영어 같은 언어
  - 규칙을 정해 만든 프로그래밍 언어와 달리 쓰면서 자연스럽게 만들어진 언어
  - 같은 말이 상황에 따라 다른 뜻을 갖고 문법에도 예외가 많음
- 자연어처리(NLP): 컴퓨터가 자연어를 이해·처리하게 하는 기술
  - 인공지능의 한 분야

자연어처리(NLP: Natural Language Processing)

자연어를 컴퓨터가 이해하고 처리하도록 만드는 기술

자연어 이해와 자연어 생성



번역기·챗봇은 이해와 생성을 모두 수행한다

그림. 자연어 이해와 자연어 생성

- 자연어 이해(NLU): 형태소·구문·의미를 분석해 언어의 뜻을 파악
- 자연어 생성(NLG): 처리 결과를 사람이 읽을 수 있는 문장으로 만듦
- 번역기는 원문의 뜻을 파악한 뒤 다른 언어로 문장을 만들므로 두 작업을 모두 수행

자연어처리의 발전 과정

- 1950년 앨런 튜링(Alan Turing)의 튜링 테스트(Turing test) 제안으로 본격 논의
- 규칙 기반(rule-based): 사람이 문법과 대화 규칙을 직접 만들어 입력
  - 1966년 조지프 바이젠바움(Joseph Weizenbaum)의 엘리자(ELIZA), 규칙과 예외를 다 적어 넣기에는 한계
- 통계 기반(statistical): 단어의 사용 빈도와 확률을 계산
  - N-그램(N-gram) 모델, 구글 번역의 통계 기반 기계번역(2006~2016년)
- 신경망 기반(neural network-based): 2010년대 주류, 의미와 문맥을 스스로 학습
  - 순환신경망 → 2017년 트랜스포머(transformer) → 거대언어모델(LLM)

자연어처리 기술의 발전 흐름

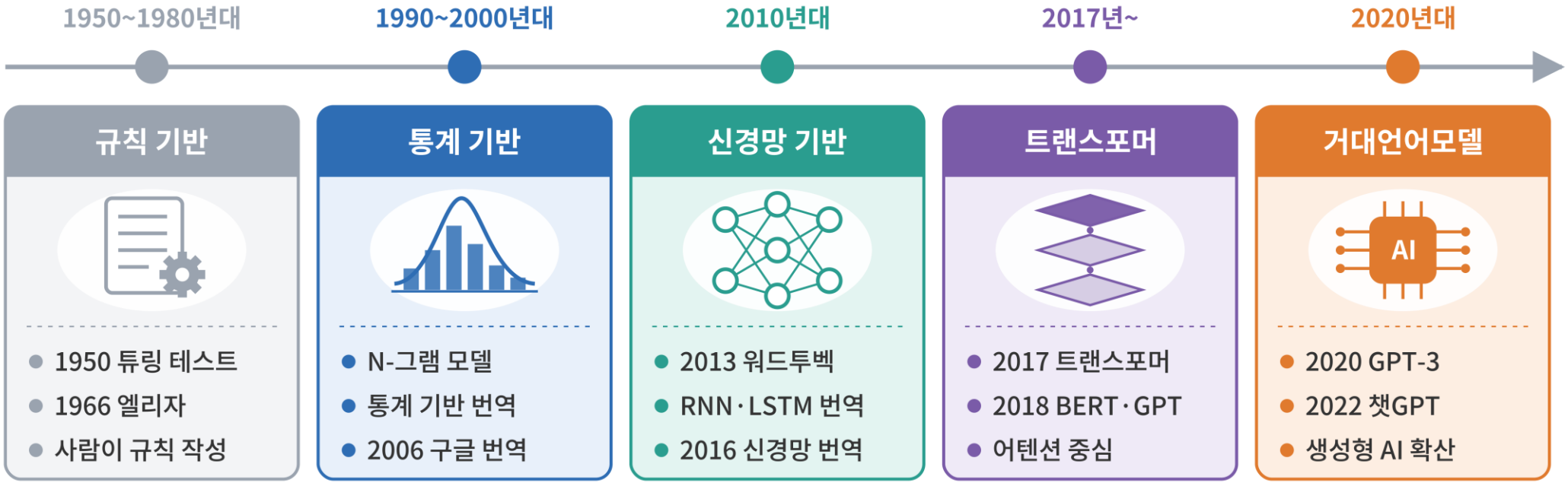


그림. 자연어처리 기술의 발전 흐름

■ 규칙 기반 → 통계 기반 → 신경망 기반 → 트랜스포머 → 거대언어모델

자연어처리의 활용 분야

- 검색
- 기계번역(machine translation)
- 챗봇과 음성 비서의 대화 처리
- 감성 분석(sentiment analysis): 글에 담긴 감정이나 의견을 판별
- 문서 요약
- 챗GPT(ChatGPT) 같은 거대언어모델도 자연어처리 기술을 바탕으로 만들어짐

자연어처리의 주요 활용 분야

분야	하는 일	활용 예
검색	질문의 의도를 파악해 관련 문서를 찾는다	포털·쇼핑몰 검색
기계번역	한 언어의 문장을 다른 언어로 옮긴다	파파고, 구글 번역
대화 처리	질문을 이해하고 알맞은 답을 만든다	챗봇, 음성 비서, 챗GPT
감성 분석	글에 담긴 긍정·부정 의견을 판별한다	상품 후기·댓글 분석
문서 요약	긴 문서의 핵심을 짧게 정리한다	뉴스·회의록 요약

표. 자연어처리의 주요 활용 분야

순차 데이터와 순환신경망

- 순차 데이터(sequential data): 단어의 순서가 의미를 결정
  - "강아지가 고양이를 쫓는다" ↔ "고양이가 강아지를 쫓는다"
- 일반적인 인공신경망은 입력을 한꺼번에 받아 순서를 반영하지 못함
- 순환신경망(RNN): 문장을 한 단어씩 차례로 입력받음
  - 시점(time step): 단어를 하나 읽는 각 단계
  - 은닉 상태(hidden state): 지금까지 읽은 내용을 요약한 기억(벡터)
- 자신의 출력을 다시 입력으로 되돌려 쓰는 구조 → '순환'
- 시계열 데이터(time-series data) 분석에도 널리 쓰임

순환신경망(RNN: Recurrent Neural Network)

은닉 상태를 다음 시점으로 넘기며 문장을 한 단어씩 처리하는 신경망

순환신경망의 구조와 펼친 모습

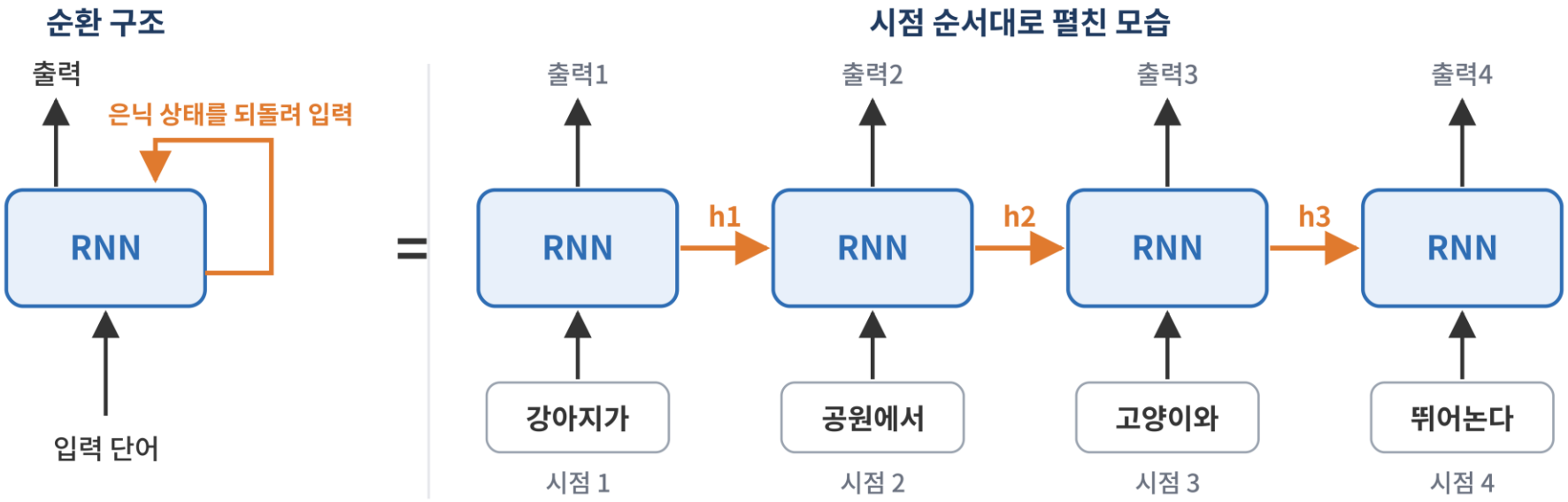


그림. 순환신경망의 구조와 펼친 모습

- 왼쪽: 은닉 상태를 되돌려 입력하는 순환 구조
- 오른쪽: 시점 순서대로 펼친 모습, 같은 RNN을 반복 사용하며 은닉 상태(h)로 앞의 기억을 다음 시점에 넘김

장기 의존성 문제

- 문장이 길어질수록 앞쪽 단어의 정보가 뒤쪽까지 온전히 전달되지 못함
  - 은닉 상태가 갱신을 거듭할수록 앞의 정보가 희미해짐
- 기울기 소실(vanishing gradient): 오차 신호(기울기)가 앞쪽 시점으로 갈수록 같은 값이 반복해 곱해져 사라짐
  - 1991년 제프 호흐라이터(Sepp Hochreiter)가 처음 분석
- 가까운 단어 사이 관계는 잘 학습하지만 멀리 떨어진 관계는 학습하지 못함
- 장기 의존성(long-term dependency): 멀리 떨어진 단어 사이의 관계

순환신경망의 장기 의존성 문제



그림. 순환신경망의 장기 의존성 문제

- 은닉 상태 속 '프랑스' 정보가 뒤로 갈수록 점점 희미해짐
- 빈칸(프랑스어)의 단서가 멀리 떨어져 있음 = 장기 의존성

[IT Focus]

숫자로 보는 기울기 소실

- 시점을 거슬러 갈 때마다 기울기에 특정 값이 곱해짐
- 곱하는 값이 0.9이면 n개 시점 뒤 신호 크기는 0.9^n
  - $0.9^{10} \approx 0.35$, $0.9^{50} \approx 0.005$, $0.9^{100} \approx 0.00003$
  - 10개 시점 뒤 약 35%, 50개 뒤 약 0.5%, 100개 뒤 사실상 0
- 기울기 폭주(exploding gradient): 1보다 크면 기하급수적으로 커짐
  - $1.1^{50} \approx 117$ → 원래의 100배 이상
- 기울기 소실과 함께 RNN이 긴 문장을 안정적으로 학습하기 어려운 근본 원인

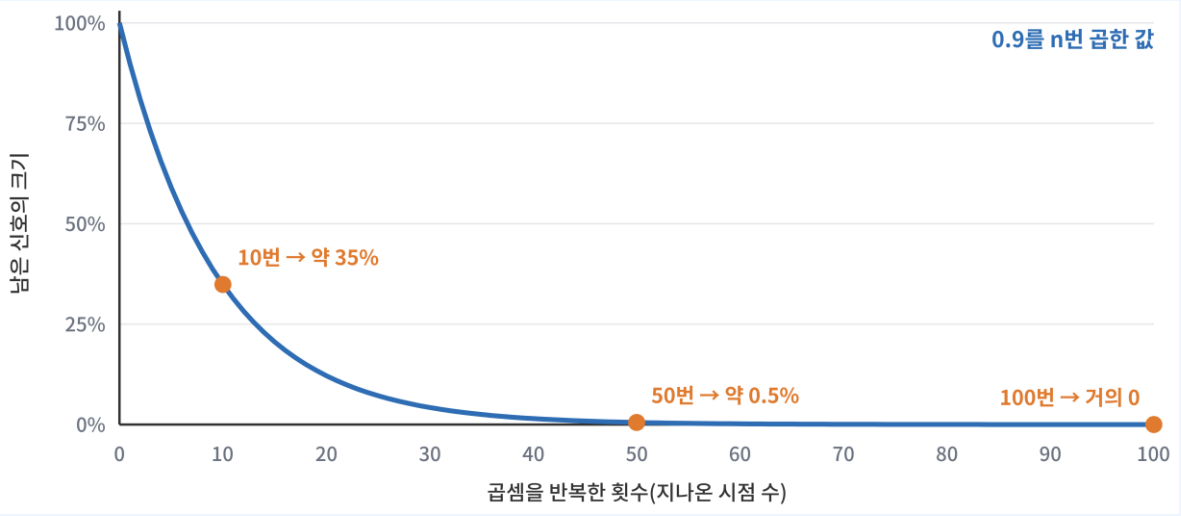


그림. 곱셈 반복 횟수에 따른 신호 크기 변화

장기 의존성 문제의 대안

- 장단기 기억(LSTM): 1997년 호흐라이터와 위르겐 슈미트후버(Jürgen Schmidhuber) 제안
  - 셀 상태(cell state): 은닉 상태와 별도로 장기 기억을 저장
  - 게이트(gate): 정보를 받아들이고 지우고 내보낼 정도를 0~1 값으로 조절
- 게이트 순환 유닛(GRU): 2014년 조경현(Kyunghyun Cho) 등, LSTM을 단순하게 줄인 구조
- 한계: 한 시점씩 순서대로 처리하는 구조는 그대로
  - 매우 긴 문장에서 정보 손실, 병렬 계산 어려움 → 트랜스포머 등장 배경

LSTM · GRU

게이트로 정보의 흐름을 조절해 필요한 정보를 더 오래 유지하는 RNN 구조

RNN, LSTM, GRU의 비교

구조	제안	기억 방식	특징
RNN	1980년대	은닉 상태 하나	구조가 단순하지만 긴 문장에서 정보가 손실된다
LSTM	1997년, 호흐라이터·슈미트후버	은닉 상태 + 셀 상태, 게이트 3개	긴 문장에서도 정보를 오래 유지한다
GRU	2014년, 조경현 등	은닉 상태 하나, 게이트 2개	LSTM을 단순화해 계산이 빠르다

표. RNN, LSTM, GRU의 비교

토큰화

- 토큰화(tokenization): 문장을 단어, 형태소, 또는 더 작은 조각으로 나누는 과정
  - 토큰(token): 나뉜 낱개의 단위
- 한국어는 조사와 어미가 발달한 교착어
  - 띄어쓰기로만 나누면 "강아지가", "강아지를", "강아지와"가 모두 다른 단어
  - 뜻을 가진 가장 작은 단위인 형태소로 나누는 방법을 많이 씀
- 부분단어(subword): 자주 함께 나타나는 글자 조각을 하나의 토큰으로 묶음
- 대표 프로그램: 은전한닢 프로젝트(mecab-ko), 구글 SentencePiece, 오픈AI tiktoken

토큰화 방식에 따른 토큰 나누기 비교



그림. 토큰화 방식에 따른 토큰 나누기 비교

- 띄어쓰기 단위 4개 · 형태소 단위 8개 · 부분단어 단위 14개
- 부분단어 단위는 오픈AI 토크나이저(platform.openai.com/tokenizer)로 실제로 나눈 결과, 토큰 수 직접 확인 가능

워드 임베딩

- 번호를 붙이는 방법에는 뜻이 담기지 않음
  - "강아지" 15번, "고양이" 16번이라도 뜻이 비슷한 것은 아님
  - 번호끼리 더하거나 빼도 의미 없음
- 워드 임베딩(word embedding): 단어를 여러 숫자로 된 벡터로 나타내 뜻이 비슷한 단어일수록 비슷한 벡터를 갖게 함
- 말뭉치(corpus)로 학습: 어휘 집합(vocabulary)을 만들고 비슷한 문맥의 단어일수록 벡터가 가까워지게 조정
- 워드투벡(Word2Vec): 2013년 토마시 미콜로프(Tomáš Mikolov) 등 구글 연구진
  - 단어와 주변 단어의 관계를 예측해 보통 100~300차원 벡터를 얻음

임베딩 벡터

워드 임베딩으로 얻은 단어의 벡터

뜻이 비슷한 단어끼리 모이는 임베딩 공간

- 동물·탈것·과일처럼 비슷한 문맥의 단어끼리 가까이 모임
- 실제 임베딩 벡터는 100~300차원, 그림은 2차원 평면으로 단순화

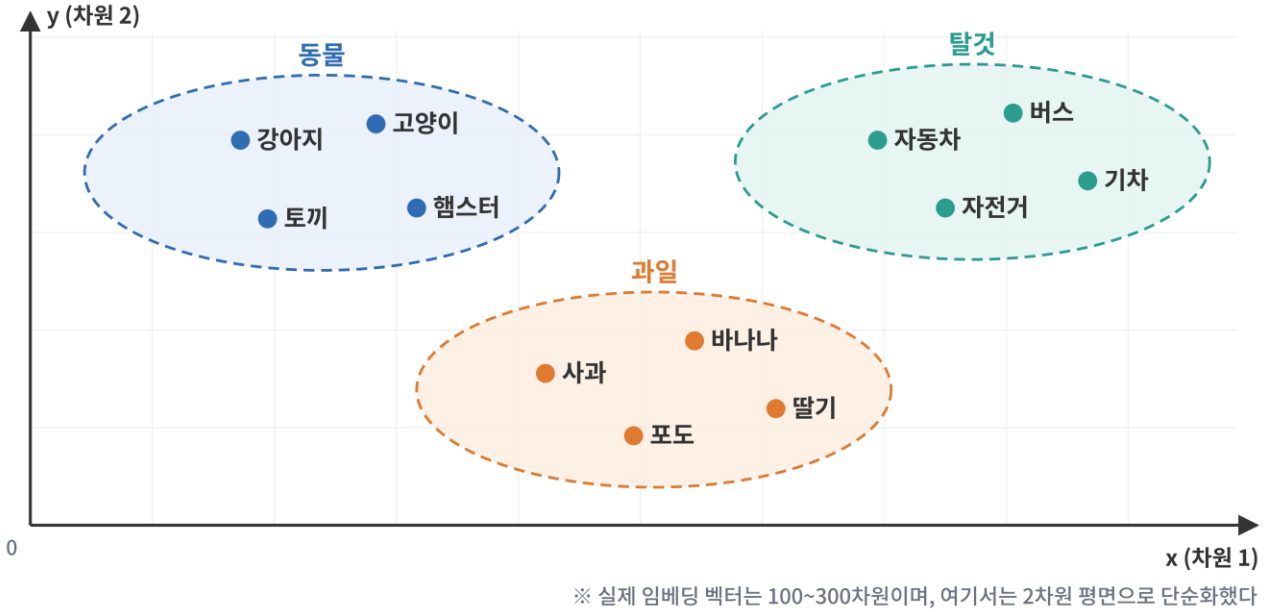


그림. 뜻이 비슷한 단어끼리 모이는 임베딩 공간

임베딩 벡터의 연산

- 임베딩 벡터는 덧셈과 뺄셈으로 단어 사이의 관계를 나타낼 수 있음
- "왕(king)" − "남자(man)" + "여자(woman)" → "여왕(queen)"과 가장 가까움 (워드투벡 연구)
  - king − man: '남성' 성질이 빠지고 '왕족' 성질만 남음
  - + woman: '여성 왕족', 즉 queen
- 의미 관계까지 벡터 공간의 방향과 거리로 담아냄
- 트랜스포머와 거대언어모델에서도 입력을 처리하는 첫 단계로 쓰임

king − man + woman ≈ queen

① king − man: 남성 성분이 빠지고 '왕족' 방향만 남음

② + woman: woman에서 같은 방향으로 같은 거리만큼 이동

③ 도착 위치에서 가장 가까운 단어: queen

- 좌표는 Word2Viz(GloVe 50차원 벡터를 두 축에 투영) 화면에서 읽은 근삿값

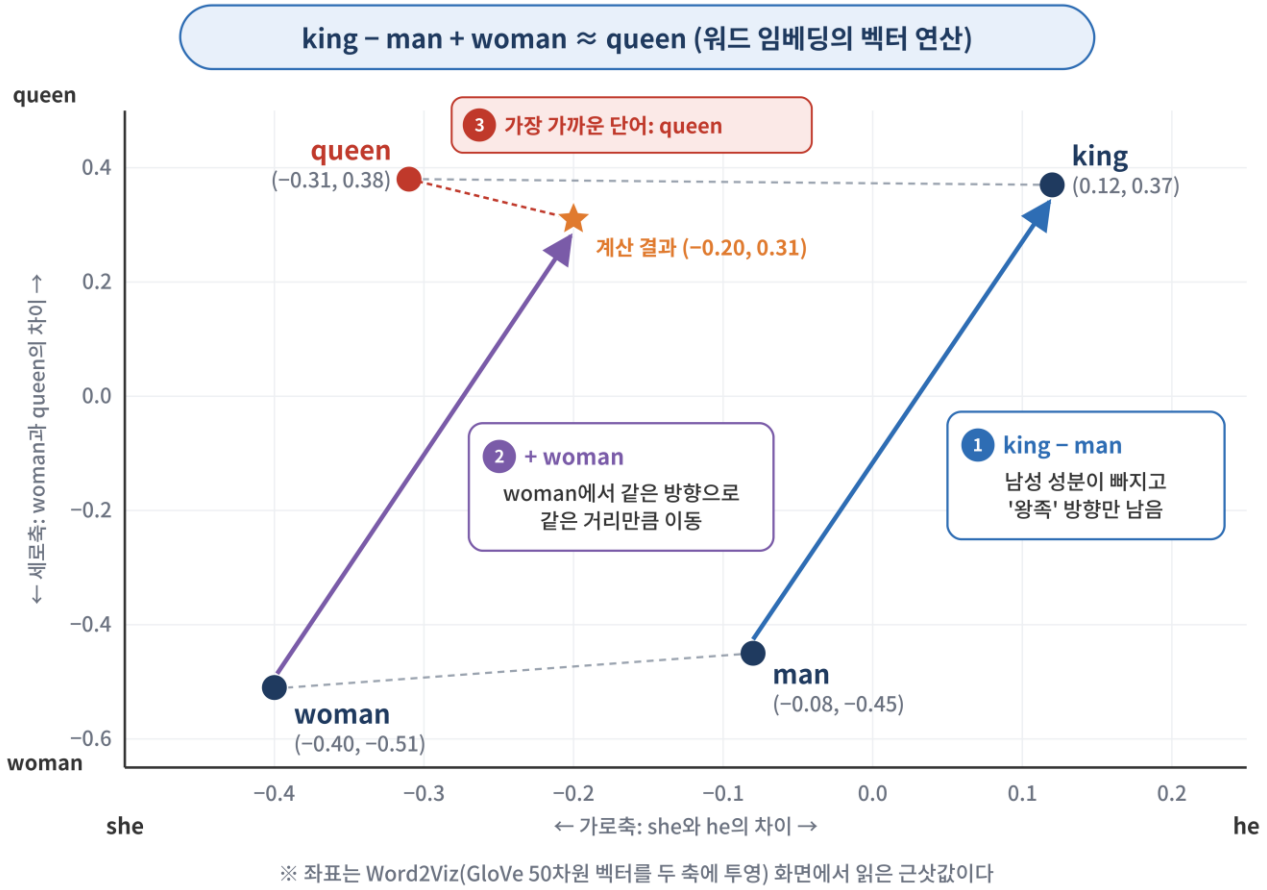


그림. 임베딩 벡터의 연산: king − man + woman ≈ queen

[IT Lab]

Word2Viz로 단어 벡터 연산 체험하기

① Word2Viz(lamyiowce.github.io/word2viz) 접속

② "What do you want to see?" 목록에서 "Gender analogies" 선택

③ king·queen·man·woman 같은 단어들이 2차원 평면에 나타남

- 가로축 she–he, 세로축 woman–queen의 차이 기준

④ 성별만 다른 단어 쌍(king–queen, man–woman, uncle–aunt)이 비슷한 기울기의 선으로 이어짐

- 성별이라는 관계가 임베딩 공간에서 일정한 방향으로 나타남

⑤ "Modify words"에 단어·단어 쌍 추가, "X axis"·"Y axis" 단어를 바꿔 관계 확인

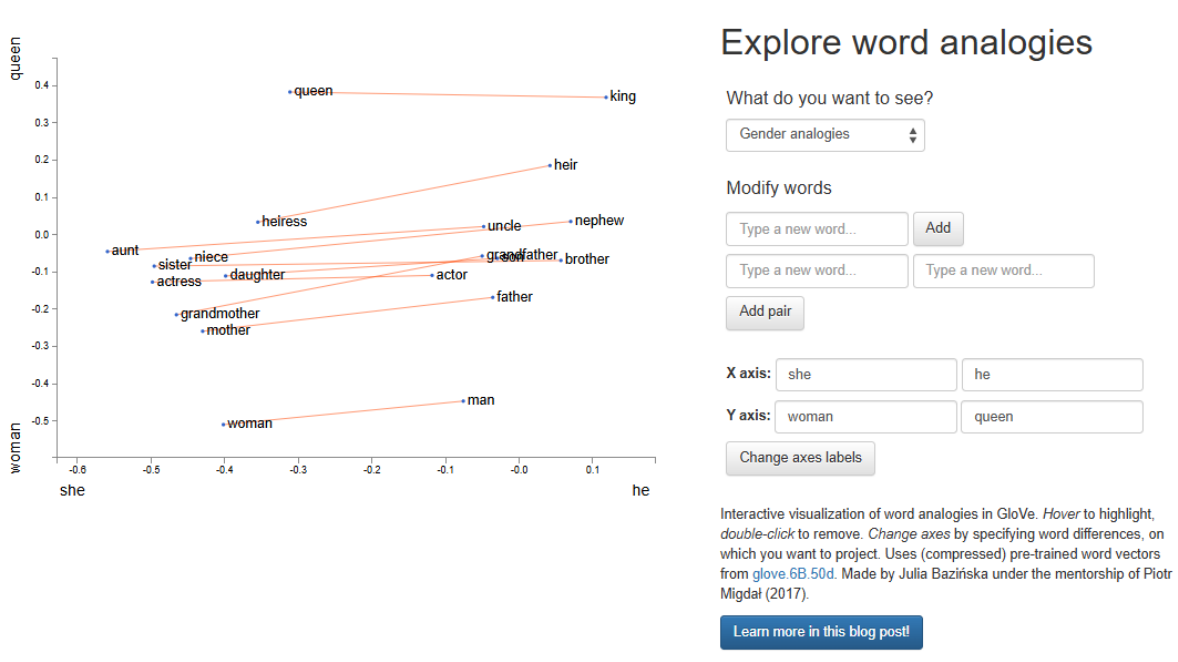


그림. Word2Viz 실행 화면(https://lamyiowce.github.io/word2viz)

정적 임베딩의 한계

- "배": 문맥에 따라 신체 부위, 과일, 선박처럼 서로 다른 뜻
- 단어마다 벡터가 하나씩 정해져 어떤 문장에서든 항상 같은 벡터
- 중의성(ambiguity)을 구분하지 못함
- 정적 임베딩(static embedding): 문맥과 관계없이 고정된 벡터
- 트랜스포머는 문맥에 따라 벡터를 다시 계산 → 같은 단어도 다른 벡터로 표현

정적 임베딩(static embedding)

문맥과 관계없이 고정된 벡터

뜻이 달라도 같은 벡터를 갖는 “배”

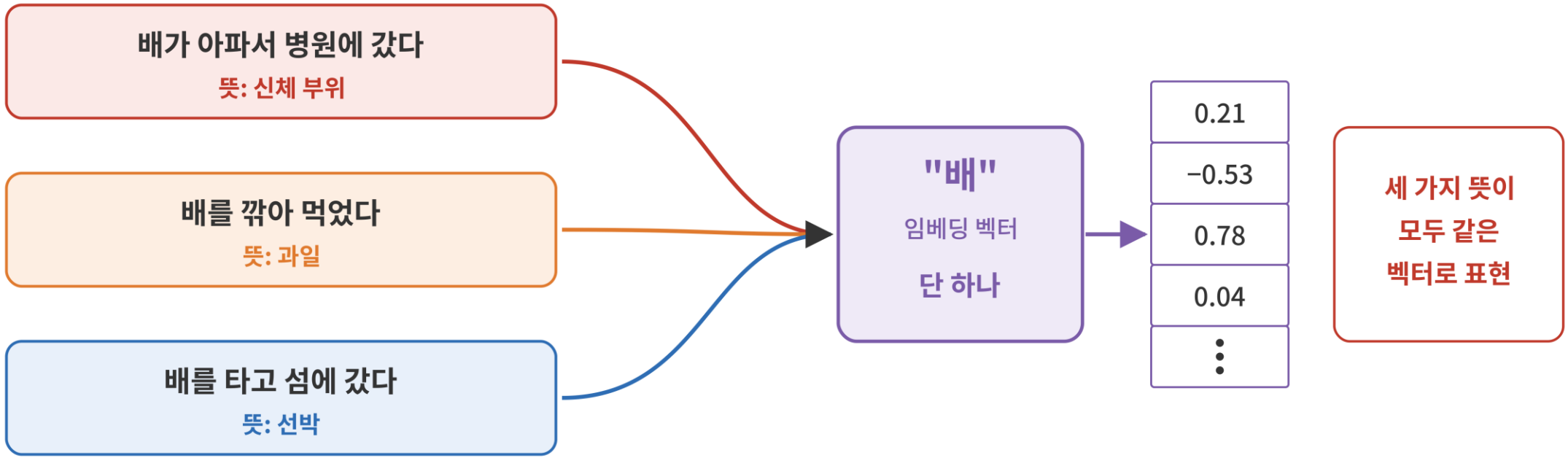


그림. 정적 임베딩의 한계: 뜻이 달라도 같은 벡터를 갖는 “배”

- 신체 부위·과일·선박의 “배”가 모두 하나의 임베딩 벡터로 표현됨

7.1 정리

1

자연어처리(NLP)

자연어 이해(NLU)와 자연어 생성(NLG),
규칙 기반 → 통계 기반 → 신경망
기반으로 발전

2

순환신경망(RNN)

은닉 상태로 문장을 한 단어씩 처리,
기울기 소실로 장기 의존성 문제 발생

3

LSTM · GRU

게이트로 정보를 오래 유지하지만
순차 처리의 한계는 그대로

4

토큰화

문장을 단어·형태소·부분단어 단위의
토큰으로 나눔

5

워드 임베딩

뜻이 비슷한 단어일수록 비슷한 벡터,
벡터 연산으로 의미 관계 표현

6

정적 임베딩의 한계

문맥과 관계없이 고정된 벡터라
중의성을 구분하지 못함

7.2

트랜스포머와 인코더

7.2.1 트랜스포머의 등장
7.2.2 트랜스포머의 입력 준비
7.2.3 셀프 어텐션의 원리
7.2.4 인코더 블록과 구조

어텐션의 등장

- 2010년대 중반 신경망 번역 모델: 두 개의 RNN을 이어 붙인 구조
- 인코더(encoder): 원문을 끝까지 읽어 마지막 은닉 상태를 원문 전체의 요약으로 삼음
- 디코더(decoder): 요약 벡터 하나만 보고 번역문을 한 단어씩 만듦
  - 방금 만든 단어를 다음 단계의 입력으로 다시 씀
- 문제: 인코더와 디코더를 잇는 통로가 크기가 정해진 벡터 하나뿐
  - 문장이 길수록 앞부분 정보가 빠져 번역 품질이 떨어짐

RNN 인코더-디코더 번역 모델

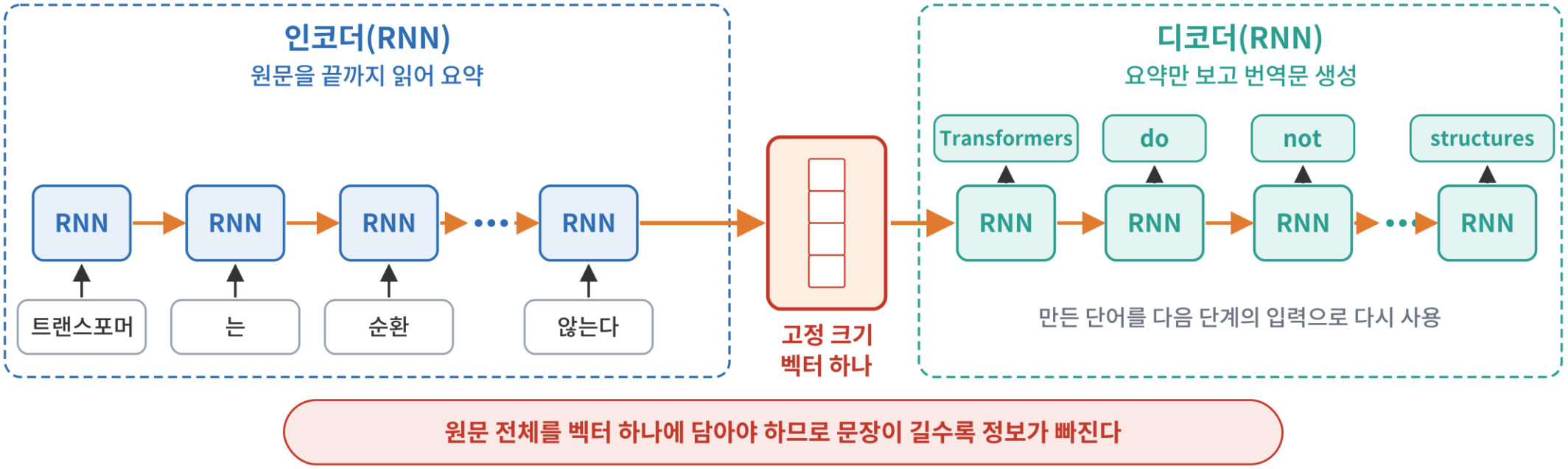


그림. RNN 인코더-디코더 번역 모델

- 인코더(RNN)는 원문을 끝까지 읽어 고정 크기 벡터 하나로 요약
- 디코더(RNN)는 요약만 보고 번역문을 생성 → 문장이 길수록 정보가 빠짐

어텐션의 등장 (2)

- 2014년 바다나우(Dzmitry Bahdanau), 조경현, 벤지오(Yoshua Bengio) 제안
  - 번역 단어마다 원문 전체를 다시 살펴보고 관련 깊은 단어에 더 주목
- 어텐션(attention): 지금 필요한 정보와 관련 깊은 것에 더 큰 비중을 두어 참고
- 어텐션 가중치(attention weight): 0과 1 사이, 모두 더하면 1인 비율
- 인코더는 원문 토큰마다 만든 은닉 상태를 모두 남겨 둠
- 디코더: 현재 상태와 각 은닉 상태를 비교 → 점수 → 가중치
- 문맥 벡터(context vector): 은닉 상태에 가중치를 곱해 모두 더한 벡터
  - 출력 단어마다 원문에서 주목하는 부분이 달라짐

어텐션을 이용한 RNN 인코더-디코더

- "recurrent"를 만드는 순간
- 원문의 "순환"에 가장 큰 어텐션 가중치(0.72)
- 선의 굵기와 숫자 = 어텐션 가중치

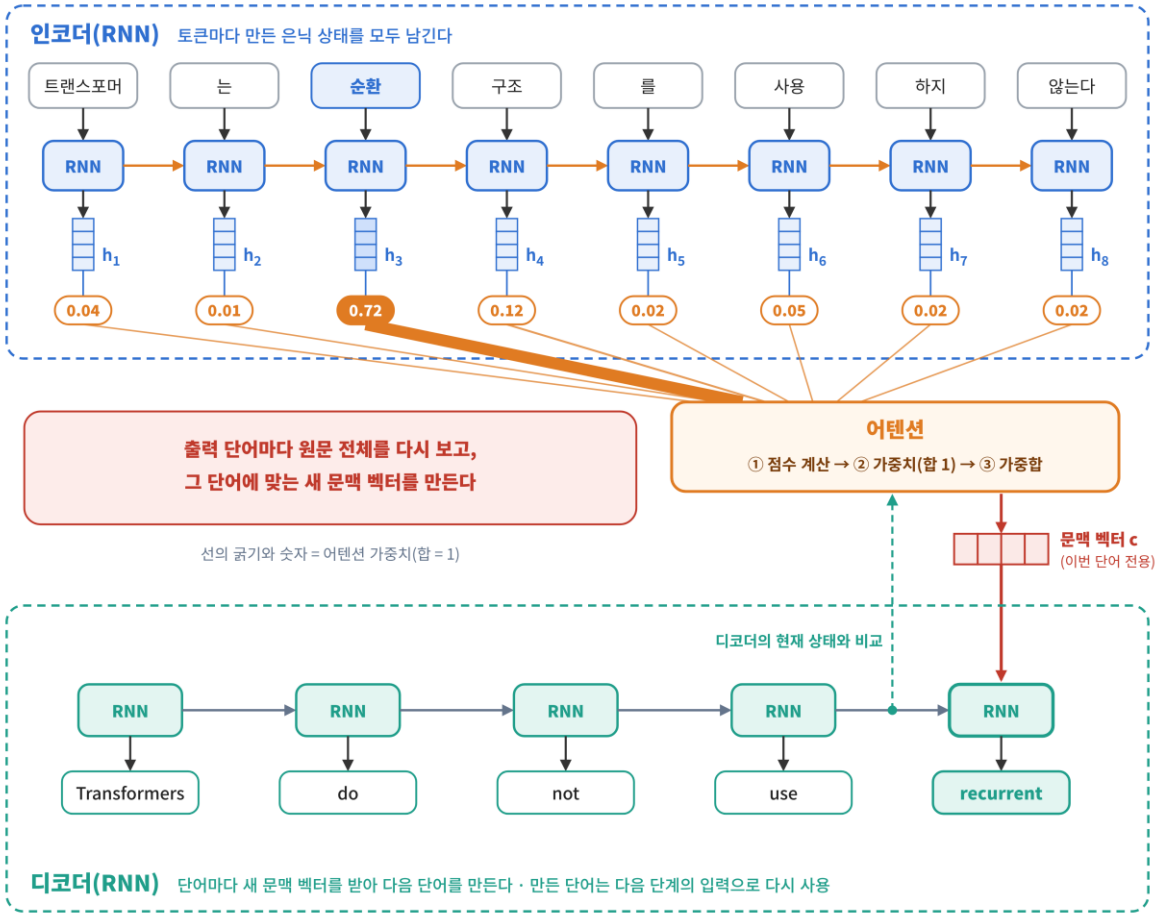


그림. 어텐션을 이용한 RNN 인코더-디코더 번역 모델

트랜스포머 원 논문

- 2017년 바스와니(Ashish Vaswani) 등 구글(Google) 연구진
  - "Attention Is All You Need(필요한 것은 어텐션뿐이다)"
  - RNN 없이 어텐션만으로 번역 모델을 만듦 → 트랜스포머(Transformer)
- 문장의 모든 단어를 한꺼번에 처리 → GPU를 잘 활용
  - 기본 모델: GPU 8개로 약 12시간 학습, 당시 최고 수준 번역 품질
- 챗GPT, 클로드(Claude), 제미나이(Gemini), 라마(Llama)의 바탕
- 비전 트랜스포머(ViT)와 멀티모달 모델로 확산

순환신경망과 트랜스포머의 처리 방식

- RNN: 한 토큰씩 순서대로 처리
  - 병렬 계산이 어려움
- 트랜스포머: 모든 토큰을 한꺼번에 처리
  - 어텐션으로 모든 토큰이 서로를 직접 참고

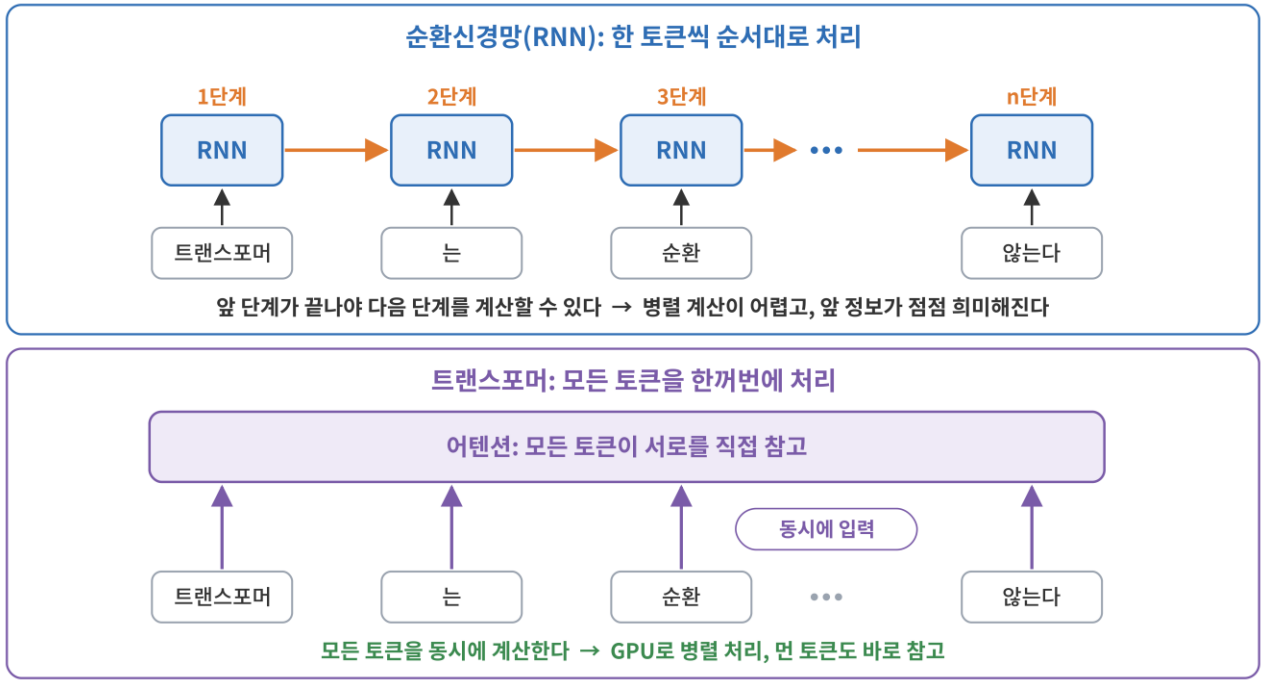


그림. 순환신경망과 트랜스포머의 처리 방식 비교

인코더와 디코더: 전체 구조 한눈에 보기

- 원 논문의 트랜스포머도 번역 모델 → 인코더와 디코더로 구성
  - 인코더: 원문을 읽고 토큰마다 문장 전체의 문맥을 담은 벡터를 만듦
  - 디코더: 이 벡터들을 참고하며 번역문을 한 토큰씩 만듦
- 원문을 벡터 하나로 요약하지 않고 토큰마다 벡터를 넘겨줌
- 블록(block): 같은 구조의 처리 단위를 여러 층으로 쌓음
  - 원 논문: 인코더 블록, 디코더 블록 각 6개
  - 블록을 올라갈수록 문맥 정보가 더 풍부해짐

트랜스포머의 전체 구조 한눈에 보기

- 인코더: 입력 준비 → 인코더 블록 ×6 → 토큰별 문맥화된 벡터
- 디코더: 입력 준비 → 블록 ×6 → 다음 토큰 예측
- 예측한 토큰을 이어 붙여 반복

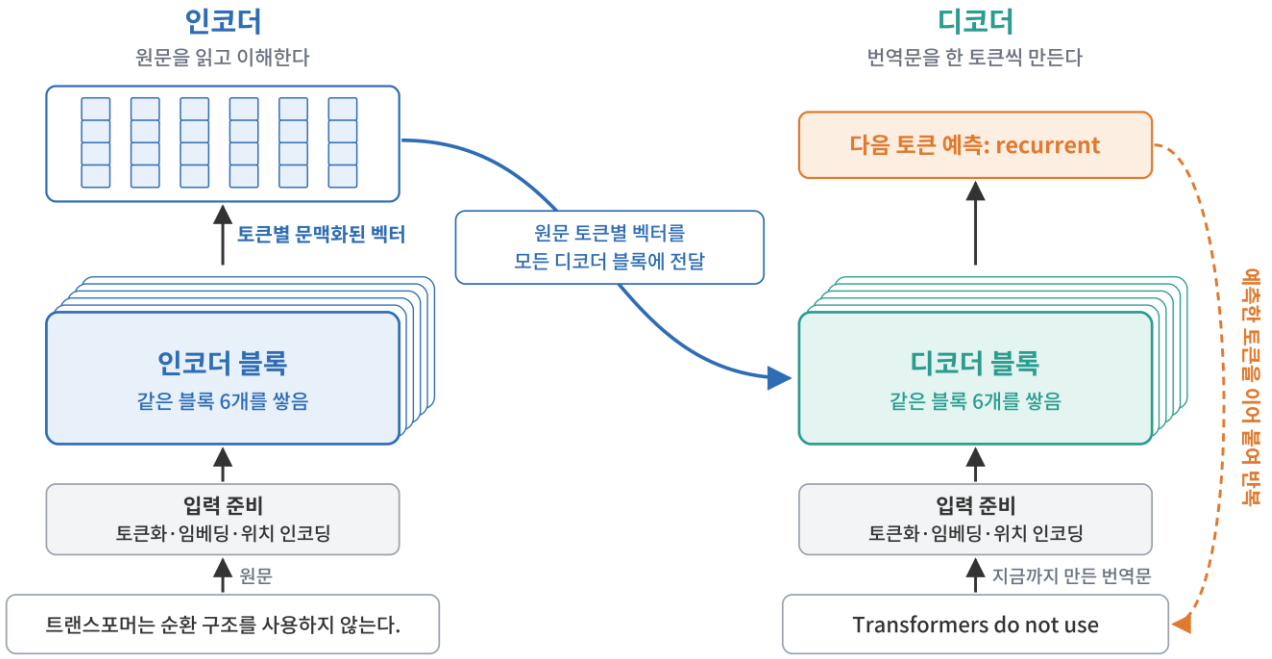


그림. 트랜스포머의 전체 구조 한눈에 보기

트랜스포머 처리의 핵심

- 핵심: 각 토큰 벡터에 문장 전체의 문맥 정보를 담아냄
- 정적 임베딩에서 "배"는 어떤 문장이든 같은 벡터
- 어텐션으로 문장 안의 다른 토큰들을 참고
  - "배가 아파서 병원에 갔다" → '신체 부위' 정보
  - "배를 타고 섬에 갔다" → '선박' 정보
- 대명사가 가리키는 명사, 동사의 주어와 목적어도 계산
- 문맥화된 벡터(contextualized vector): 문맥이 반영되어 문장마다 달라지는 벡터

문맥화된 벡터

문맥이 반영되어 문장마다 달라지는 벡터, 문장 이해나 생성에 쓰임

정적 임베딩에서 문맥화된 벡터로

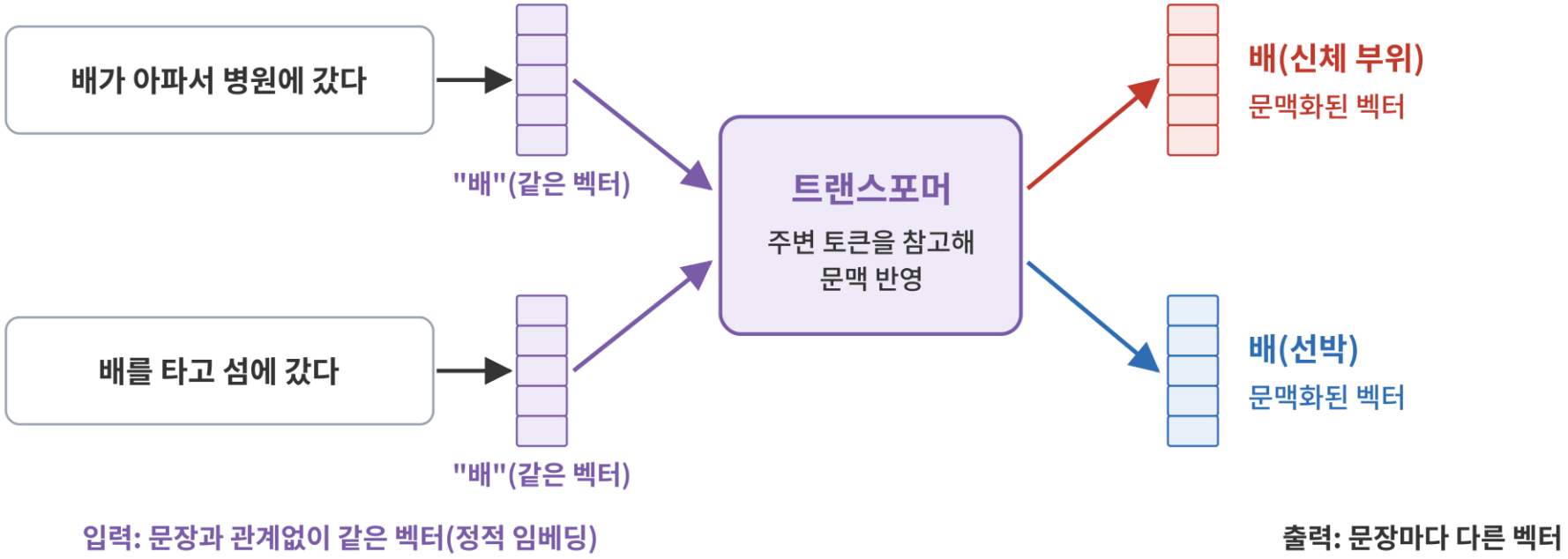


그림. 트랜스포머의 역할: 정적 임베딩에서 문맥화된 벡터로

- 입력: 문장과 관계없이 같은 벡터(정적 임베딩)
- 출력: 주변 토큰을 참고해 문장마다 다른 벡터(신체 부위·선박)

토큰화와 토큰 ID

- 문장을 토큰화해 각 토큰에 고유 번호인 토큰 ID를 붙임
- 부분단어 방식: 학습 텍스트에서 자주 붙어 나오는 글자 조각을 통계적으로 묶음
  - 언어에 관계없이 적용 가능
  - 원 논문: 영어와 독일어를 함께 학습해 약 37,000개 토큰의 어휘 사전
- 어휘 사전은 토크나이저(tokenizer)를 만들 때 미리 학습해 고정
  - 토큰화 = 사전에서 조각을 찾아 정해진 번호를 붙이는 일
- 같은 토큰은 같은 토큰 ID: "사용"과 "."

예문의 토큰화 결과

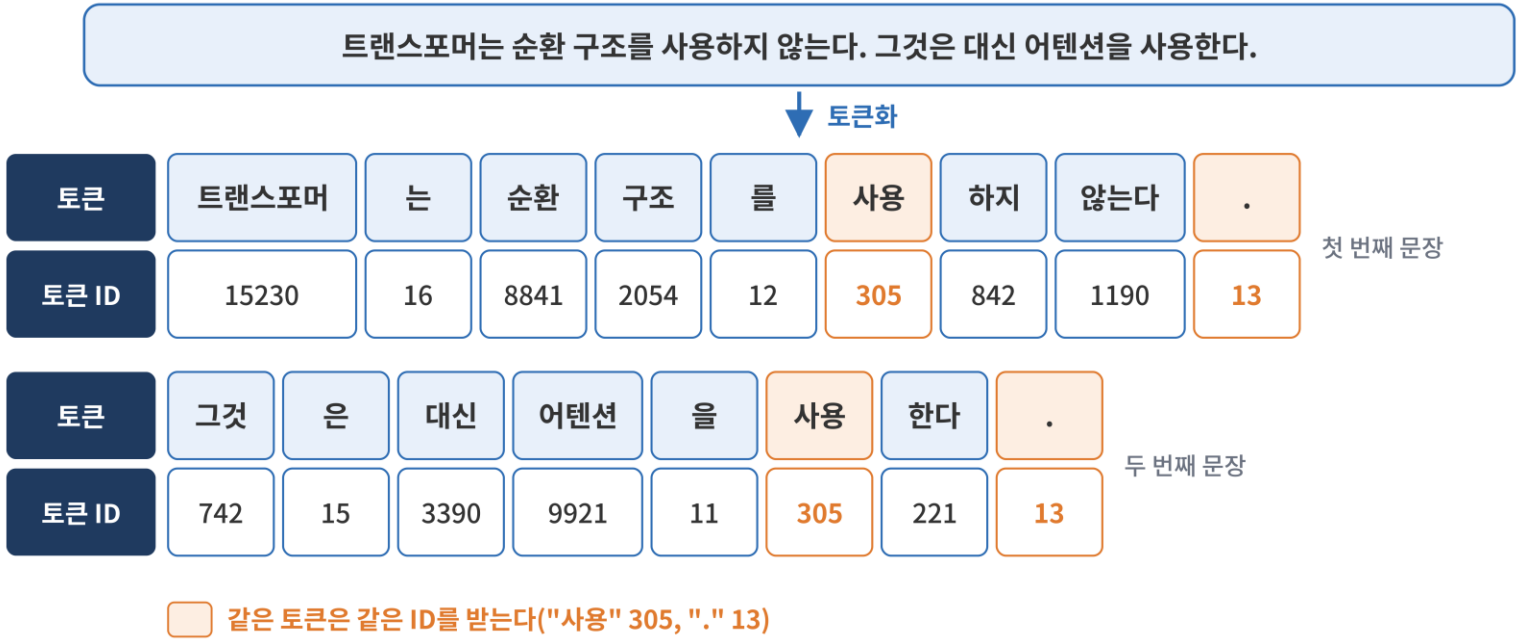


표. 예문의 토큰화 결과(ID는 이해를 돕기 위해 가정한 예시 값)

- 두 번 나오는 "사용"은 305, 마침표 "."는 13으로 같은 ID
- ID 값은 이해를 돕기 위해 가정한 예시 값

토큰 임베딩

- 토큰 ID는 사전 속 위치를 가리키는 번호일 뿐 뜻이 없음
- 임베딩 행렬(embedding matrix): 토큰마다 행을 하나씩 배정해 벡터를 담은 표
  - 행의 수 = 어휘 사전의 토큰 수(원 논문 약 37,000개)
- 토큰 임베딩: 토큰 ID를 행 번호로 삼아 해당 행을 읽어 옴
  - "트랜스포머"(ID 15230)의 벡터는 15230번 행
- 임베딩 차원(embedding dimension): 벡터 하나에 든 숫자의 개수(원 논문 512)
- 약 37,000행 × 512열 = 약 1,894만 개의 매개변수(parameter)

토큰 ID로 임베딩 행렬에서 벡터 찾기

- 처음부터 학습할 때 임베딩 행렬은 무작위 값에서 출발해 함께 조정됨
- 학습된 모델을 쓸 때는 학습된 임베딩 행렬을 그대로 이어받음
- 뜻이 비슷한 토큰일수록 벡터도 비슷
- 아직 정적 임베딩
  - 두 번 나오는 "사용"은 같은 벡터

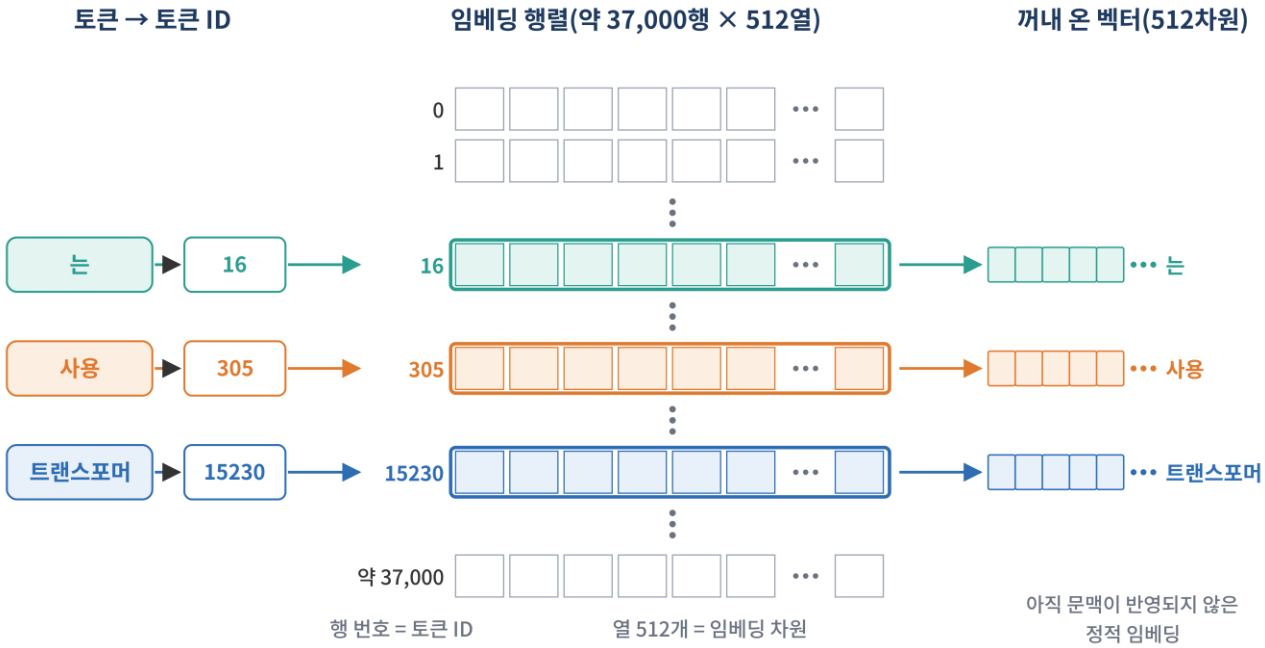


그림. 토큰 ID로 임베딩 행렬에서 벡터 찾기

위치 인코딩: 위치 정보 추가

- 트랜스포머는 모든 토큰을 동시에 처리 → 토큰의 순서를 스스로 알지 못함
  - "언어모델이 문장을 생성한다" ↔ "문장이 언어모델을 생성한다"
- 위치 인코딩(positional encoding): 위치마다 다른 위치 벡터를 규칙으로 만들어 더함
- 위치 벡터는 토큰 임베딩과 차원이 같고 위치로만 값이 정해짐
  - 같은 토큰이라도 위치가 다르면 다른 위치 벡터
- '무슨 토큰인가' + '몇 번째 토큰인가' → 인코더 첫 블록의 입력

토큰 임베딩과 위치 인코딩의 결합



그림. 토큰 임베딩과 위치 인코딩의 결합

- 같은 "언어모델"이라도 위치 0과 위치 2에 다른 위치 벡터가 더해짐
- 4차원으로 단순화한 예시 값(원 논문은 512차원)

[IT Lab] Transformer Explainer로 토큰 ID와 위치 인코딩 확인하기

- Transformer Explainer(poloclub.github.io/transformer-explainer)
  - 조지아공과대학교 Poloclub 연구팀, GPT-2(small) 모델 작동 과정 시각화

① "Transformers do not use recurrent structures" 입력 후 "Embedding" 옆 돋보기

② "Tokenization": Transform, ers, do, not, use, recurrent, structures 7개 토큰

③ "Token Embedding": 토큰 ID(Transform 41762, ers 364, do 466 등)

④ "Positional Encoding": 위치 0~6 벡터를 더함
  - 더한 결과가 첫 번째 블록의 입력

- 순서 확인: 토큰화 → 토큰 ID → 임베딩 + 위치 인코딩



그림. Transformer Explainer의 입력 준비 화면: 토큰화, 토큰 ID, 위치 인코딩

셀프 어텐션 개요

- 앞의 어텐션: 번역문의 단어가 원문의 단어를 참고
- 인코더: 한 문장 안에서 토큰들이 서로를 참고
- "트랜스포머는 순환 구조를 사용하지 않는다. 그것은 대신 어텐션을 사용한다."
  - "그것"이 가리키는 대상은 앞 문장의 "트랜스포머"를 함께 봐야 앎
  - "사용하지 않는다"는 주어 "트랜스포머", 목적어 "순환 구조"와 함께 봐야 앎
- 셀프 어텐션(self-attention): 같은 문장의 모든 토큰과의 관련성을 반영한 새 벡터를 만듦
  - 자기 자신이 속한 문장을 참고한다는 뜻에서 '셀프'

“그것”이 “트랜스포머”에 주목하는 셀프 어텐션



그림. “그것”이 “트랜스포머”에 주목하는 셀프 어텐션

- “그것”(질문하는 쪽)이 같은 문장의 모든 토큰을 참고
- “트랜스포머”에 가장 크게 주목(0.48), 선의 굵기 = 어텐션 가중치(예시 값)

질의·키·값의 이해

- 관련성을 판단해 정보를 골라 담는 데 필요한 세 가지
  - 무엇을 찾는지 → 질의(Q: Query)
  - 무엇을 기준으로 비교할지 → 키(K: Key)
  - 실제로 무엇을 전달할지 → 값(V: Value)
- 도서관 비유: "대한민국의 수도"를 찾을 때
  - 검색 질문 = 질의, 책의 목차·키워드 = 키, 책의 실제 내용 = 값
  - 서울 안내서는 많이, 과일 도감은 거의 참고하지 않음

도서관 비유로 본 질의·키·값

① 질의를 각 책의 키와 비교

② 관련성 비율 계산(서울 안내서 0.80, 파리 여행기 0.15, 과일 도감 0.05)

③ 값을 비율만큼 섞어 종합



그림. 도서관 비유로 본 질의·키·값

도서관 비유로 본 질의·키·값의 의미

구분	도서관 비유	셀프 어텐션에서의 의미	"그것"의 경우
질의(Q)	내가 찾고 싶은 내용을 담은 검색 질문	현재 토큰이 다른 토큰에서 찾으려는 정보를 나타내는 벡터	"그것"이 가리키는 대상을 찾는 질문
키(K)	질문과 맞춰 보는 각 책의 목차나 키워드	다른 토큰의 질의와 비교해 관련도를 계산하는 데 쓰는 각 토큰의 벡터	"트랜스포머"의 키가 질문과 가장 잘 맞음
값(V)	각 책에서 실제로 참고할 내용	관련도에 따라 가져와 반영할 각 토큰의 정보 벡터	"트랜스포머"의 정보를 가장 많이 가져옴
어텐션 결과	관련 있는 책의 내용을 더 큰 비중으로 종합한 정보	질의와 키로 구한 비중만큼 값들을 섞어 합한 새로운 벡터	트랜스포머를 가리킨다는 정보가 담긴 "그것"의 새 벡터

표. 도서관 비유로 본 질의·키·값의 의미

모든 토큰이 질의·키·값을 갖는 셀프 어텐션

- 셀프 어텐션은 모든 토큰에서 동시에
- 모든 토큰은 질문하는 쪽이자 받는 쪽
  - 질의(Q)와 키(K)·값(V)을 모두 가짐
- 같은 입력 벡터에 다른 가중치 행렬을 곱함

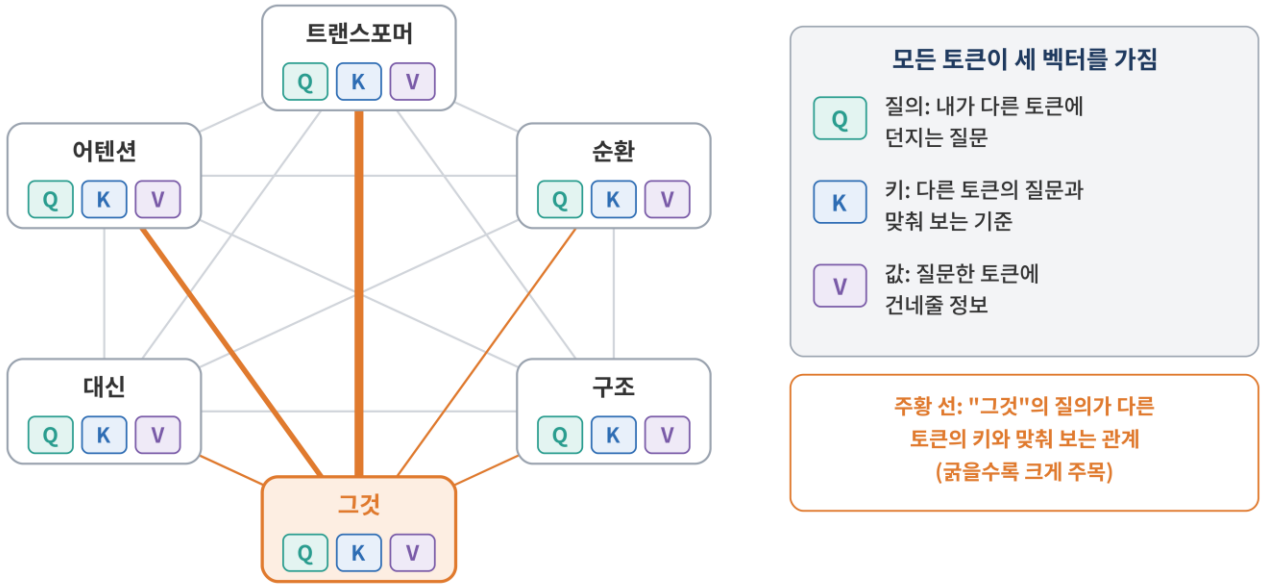


그림. 모든 토큰이 질의·키·값을 갖는 셀프 어텐션

어텐션 가중치와 소프트맥스

- 어텐션 점수(attention score): 한 토큰의 질의와 모든 토큰의 키가 맞는 정도
  - 방향이 비슷할수록 크며, 음수 가능·크기 제한 없음 → 비율로 바꿔야 함
- 소프트맥스(softmax): 점수를 0~1 사이, 합이 1인 값으로 바꾸는 함수
  ① 각 점수만큼 e(약 2.718)를 거듭제곱 → 항상 양수, 클수록 급격히 커짐
  ② 모두 더한 합으로 나누어 비율로 만듦
- 점수가 조금만 앞서도 비율 차이가 크게 벌어짐
  - 점수 2, 1, −0.5 → 약 0.69, 0.25, 0.06

소프트맥스(softmax) 함수의 역할

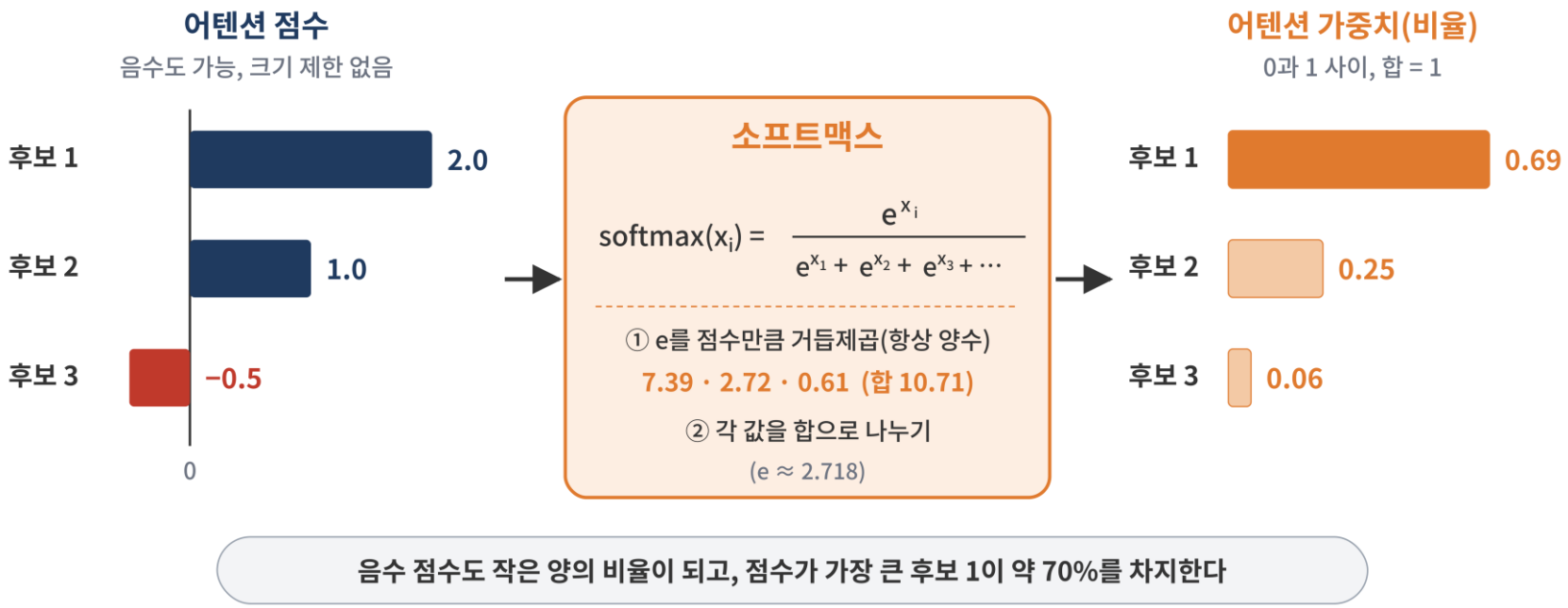


그림. 소프트맥스(softmax) 함수의 역할

① e를 점수만큼 거듭제곱: 7.39 · 2.72 · 0.61(합 10.71)

② 합으로 나누기 → 0.69 · 0.25 · 0.06, 음수 점수도 작은 양의 비율

어텐션 점수에서 문맥화된 벡터까지

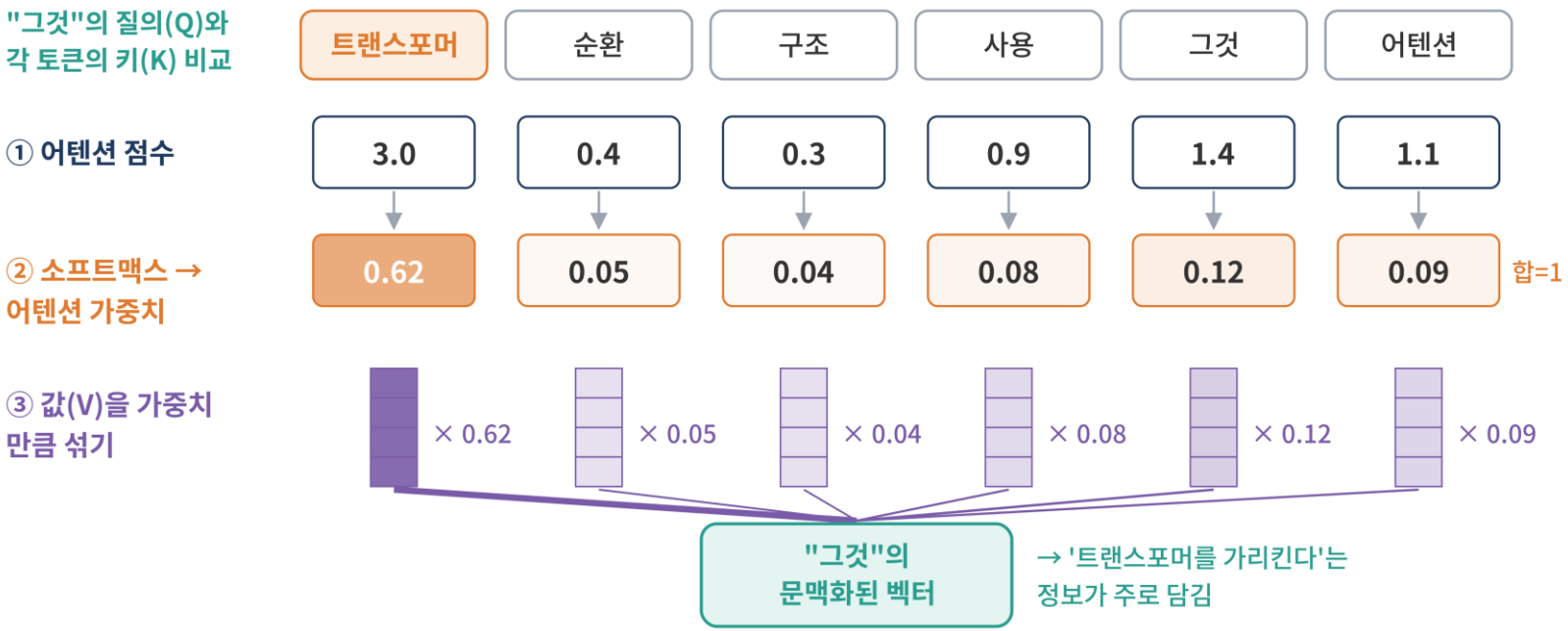


그림. 어텐션 점수에서 문맥화된 벡터까지("그것"의 경우)

- 소프트맥스를 거친 비율 = 어텐션 가중치(디코더의 다음 토큰 선택에도 쓰임)
- 각 토큰의 값에 가중치를 곱해 모두 더함 → 관련 깊은 토큰의 정보가 주로 담김
- "그것" → '트랜스포머를 가리킨다'는 정보의 문맥화된 벡터

어텐션 맵: 모든 토큰 쌍의 어텐션 가중치

- 모든 토큰이 동시에 계산 → n×n개 가중치
- 어텐션 맵(attention map): 행 = 질문하는 토큰, 열 = 참고되는 토큰
  - 값의 크기를 색의 진하기로, 한 행의 합은 1
- 문장이 2배 길어지면 가중치는 4배 → 긴 문장에서 계산량 급증



그림. 어텐션 맵: 모든 토큰 쌍의 어텐션 가중치(n × n)

어텐션 가중치와 소프트맥스 (2)

- 질의·키·값을 만드는 가중치 행렬: 학습으로 정해지는 값
- 어텐션 가중치와 어텐션 맵: 문장이 입력될 때마다 새로 계산되는 값
- 학습하면서 관련 있는 토큰 사이에 높은 가중치가 계산되도록 가중치 행렬이 조정됨

학습되는 값 vs 계산되는 값

가중치 행렬은 학습으로 정해지고, 어텐션 가중치는 문장마다 새로 계산됨

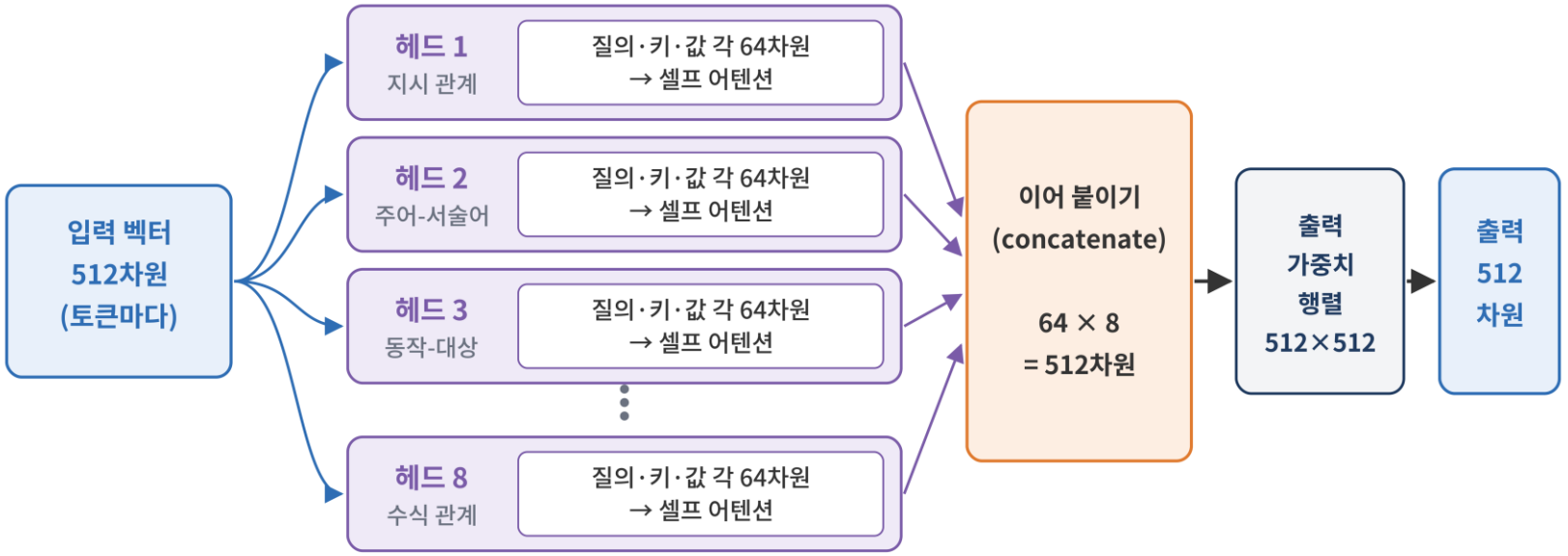
멀티 헤드 어텐션

- 토큰들의 관계는 여러 관점에서 파악할 수 있음
  - 지시 관계: "그것"이 "트랜스포머"를 가리킴
  - 주어-서술어 관계: "사용하지 않는다"의 주체가 "트랜스포머"
  - 동작-대상 관계: "사용한다"의 대상이 "어텐션"
  - 수식 관계: "순환"이 "구조"를 꾸밈
- 헤드(head): 자기만의 가중치 행렬로 셀프 어텐션을 한 번 수행하는 단위
- 멀티 헤드 어텐션(multi-head attention): 헤드 여러 개가 동시에 수행
  - 헤드마다 다른 관점을 맡으며, 역할 분담은 학습으로 저절로 이루어짐

멀티 헤드 어텐션 (2)

- 여러 전문가가 각자 관점에서 분석한 뒤 의견을 모으는 것과 비슷
- 원 논문: 헤드 8개, 그러나 비용은 8배가 아님
  - 각 헤드는 64차원(512의 8분의 1) 질의·키·값으로 계산
- 8개 헤드 결과(각 64차원)를 이어 붙여(concatenate) 512차원으로
- 출력 가중치 행렬(512×512)을 곱해 8개 헤드 결과를 섞음
  - 출력은 입력과 같은 512차원 → 잔차 연결에서 그대로 더할 수 있음

멀티 헤드 어텐션: 나누어 계산한 뒤 결합



헤드마다 서로 다른 가중치 행렬로 계산해 서로 다른 관계에 주목한다(원 논문: 헤드 8개)

그림. 멀티 헤드 어텐션: 여러 헤드로 나뉘어 계산한 뒤 하나로 결합

- 입력 512차원 → 헤드 8개가 각 64차원 질의·키·값으로 셀프 어텐션
- 이어 붙이기(64×8=512) → 출력 가중치 행렬(512×512) → 출력 512차원

잔차 연결과 층 정규화

- 잔차 연결(residual connection): 멀티 헤드 어텐션의 출력에 원래 입력을 다시 더함
  - 문맥 정보가 원래 벡터 위에 덧붙여져 원래 정보가 사라지지 않음
  - 기울기가 더하기 경로로 앞쪽 블록까지 잘 전달 → 기울기 소실 줄임
- 층 정규화(layer normalization): 토큰마다 평균 0, 분산 1로 값의 범위를 맞춤
  - 과목 성적을 표준점수로 바꾸어 비교하는 것과 비슷
- 항상 짝을 이루어 Add & Norm이라 부름(여기서는 '잔차 연결과 층 정규화')

잔차 연결과 층 정규화(Add & Norm)

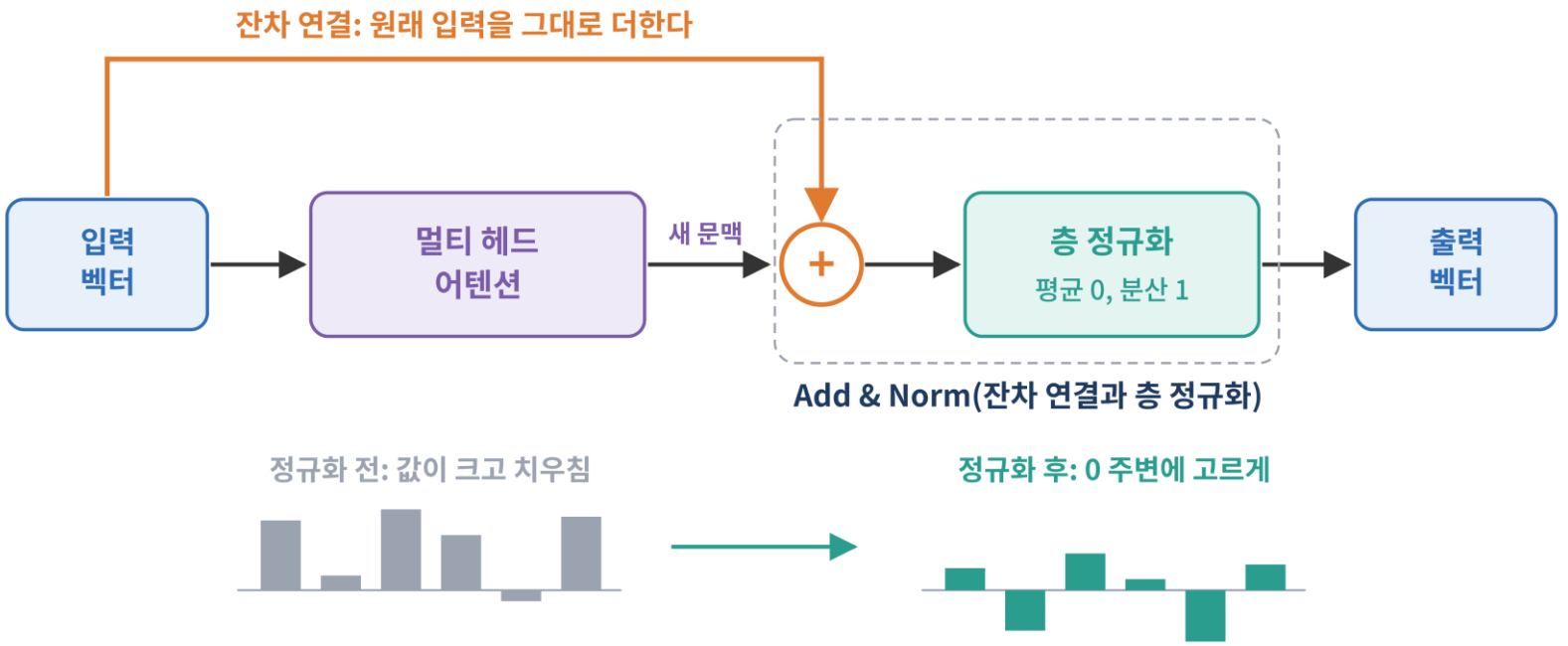


그림. 잔차 연결과 층 정규화(Add & Norm)

- 입력 벡터를 멀티 헤드 어텐션의 결과(새 문맥)에 그대로 더함
- 층 정규화로 크고 치우친 값을 0 주변에 고르게(평균 0, 분산 1)

피드포워드 신경망

- 피드포워드 신경망(FFN): 입력층 → 출력층 한 방향으로 계산이 흐르는 신경망
- 셀프 어텐션: 다른 토큰에서 정보를 '모아 오는' 단계
  - 값 벡터를 섞은 것일 뿐이라 뜻의 해석이나 새 특징 생성은 어려움
- 피드포워드 신경망: 모아 온 정보를 토큰마다 '가공하고 정리하는' 단계
  - 원 논문: 512차원 → 2,048차원으로 넓혔다가 → 512차원으로 압축
- 렐루(ReLU): 음수는 0으로, 양수는 그대로 두는 활성화 함수
  - 값을 꺾어 주어야 복잡한 관계를 표현할 수 있음

셀프 어텐션과 피드포워드 신경망의 비교

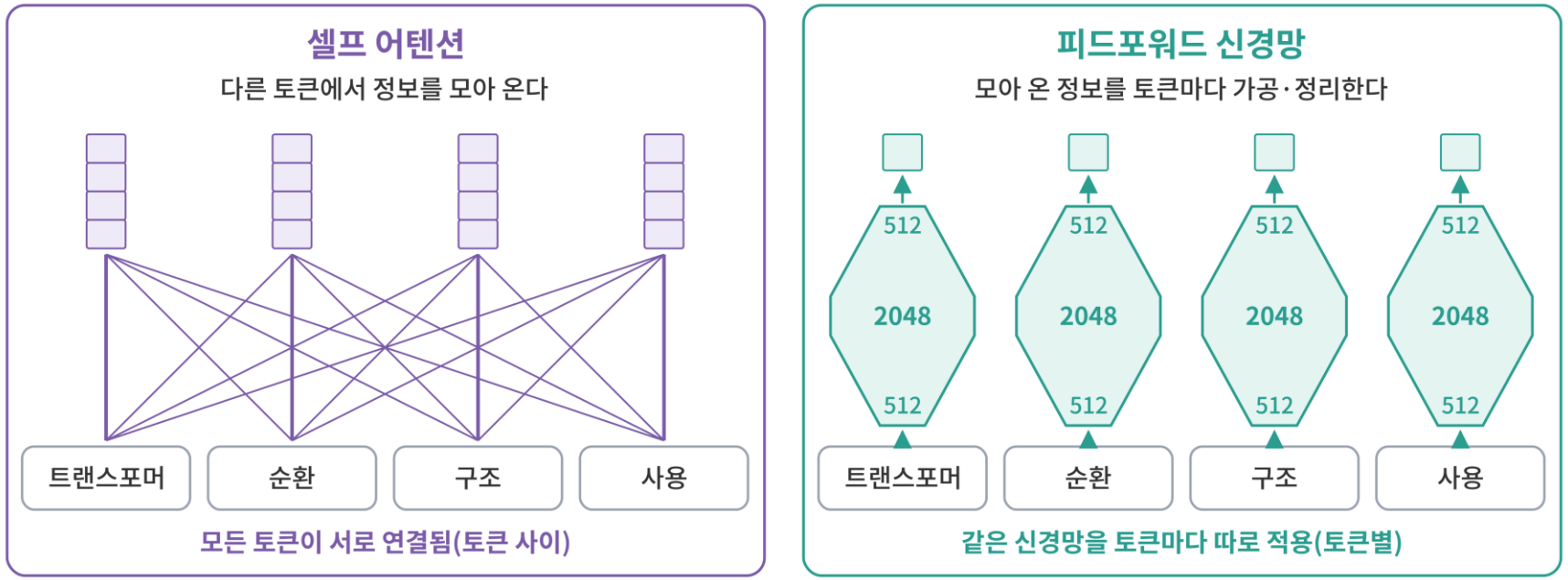


그림. 셀프 어텐션(토큰 사이)과 피드포워드 신경망(토큰별)의 비교

- 피드포워드 신경망은 다른 토큰을 참고하지 않고 토큰마다 독립적으로 처리(같은 신경망 공유)
- 트랜스포머 매개변수의 약 3분의 2, 학습한 표현 패턴을 기억하는 장치처럼 작동
- 출력에도 잔차 연결과 층 정규화 적용

인코더 블록 쌓기

- 인코더 블록(encoder block)의 구성
  - 멀티 헤드 어텐션 → 잔차 연결과 층 정규화
  - 피드포워드 신경망 → 잔차 연결과 층 정규화
- 들어온 벡터와 같은 개수·차원의 벡터를 내보냄 → 그대로 쌓을 수 있음
  - 원 논문: 블록 6개
  - 첫 블록: 토큰 임베딩 + 위치 인코딩, 두 번째부터: 앞 블록의 출력
- 블록을 거듭할수록 더 넓고 깊은 문맥 정보가 반영됨
  - 아래쪽: 형태·가까운 단어 관계, 위쪽: 문장의 의미처럼 추상적 정보
- 마지막 블록의 토큰별 문맥화된 벡터 → 디코더로 전달

인코더 블록의 구조

- 아래: 토큰 임베딩 + 위치 인코딩 = 입력 벡터(첫 번째 블록)
- 멀티 헤드 어텐션
  - → 잔차 연결과 층 정규화
- 피드포워드 신경망(토큰별)
  - → 잔차 연결과 층 정규화
- 출력 벡터 → 다음 블록 입력
  - 원 논문은 블록 6개

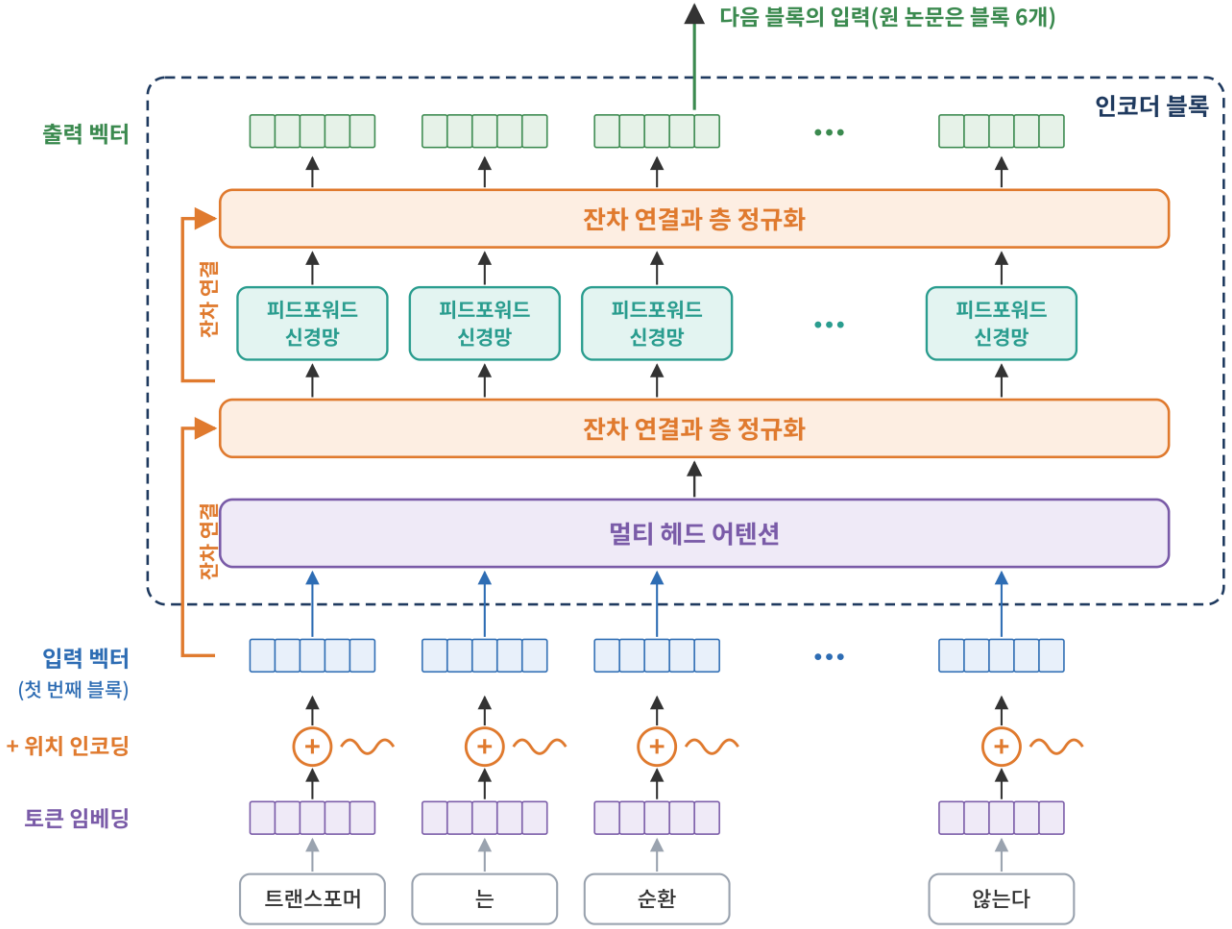


그림. 인코더 블록의 구조

7.2 정리

1

어텐션

지금 필요한 정보와 관련 깊은 것에 더 큰 비중을 두어 참고, 문맥 벡터로 RNN 번역 모델의 한계 보완

2

트랜스포머

RNN 없이 어텐션만으로 모든 토큰을 한꺼번에 처리, 인코더와 디코더 블록 각 6개

3

입력 준비

토큰화 → 토큰 ID → 토큰 임베딩 + 위치 인코딩

4

셀프 어텐션

질의·키·값으로 어텐션 점수 → 소프트맥스 → 가중합으로 문맥화된 벡터

5

멀티 헤드 어텐션

여러 헤드가 서로 다른 관계를 나누어 맡고 결과를 결합

6

인코더 블록

멀티 헤드 어텐션과 피드포워드 신경망, 각각 잔차 연결과 층 정규화

7.3

디코더와 텍스트 생성

7.3.1 디코더의 셀프 어텐션과 크로스 어텐션
7.3.2 디코더 블록과 다음 토큰 예측
7.3.3 트랜스포머의 학습·추론과 활용

디코더의 역할

- 디코더(decoder): 인코더의 문맥화된 벡터를 참고해 출력 문장을 한 토큰씩 만듦
  - 다음 토큰을 예측 → 입력 끝에 이어 붙임 → 다시 예측
  - 원문을 참고해 "Transformers", "do", "not" ...을 차례로 만듦
- 시작 토큰(BOS: Beginning Of Sequence, <BOS>)으로 시작
- 끝 토큰(EOS: End Of Sequence, <EOS>)을 예측하면 생성을 멈춤
- 입력 토큰도 토큰 임베딩과 위치 인코딩을 거쳐 벡터가 됨
- 인코더 블록과 다른 점
  - 셀프 어텐션에 마스크를 적용, 원문을 참고하는 크로스 어텐션 추가

인코더와 디코더의 관계

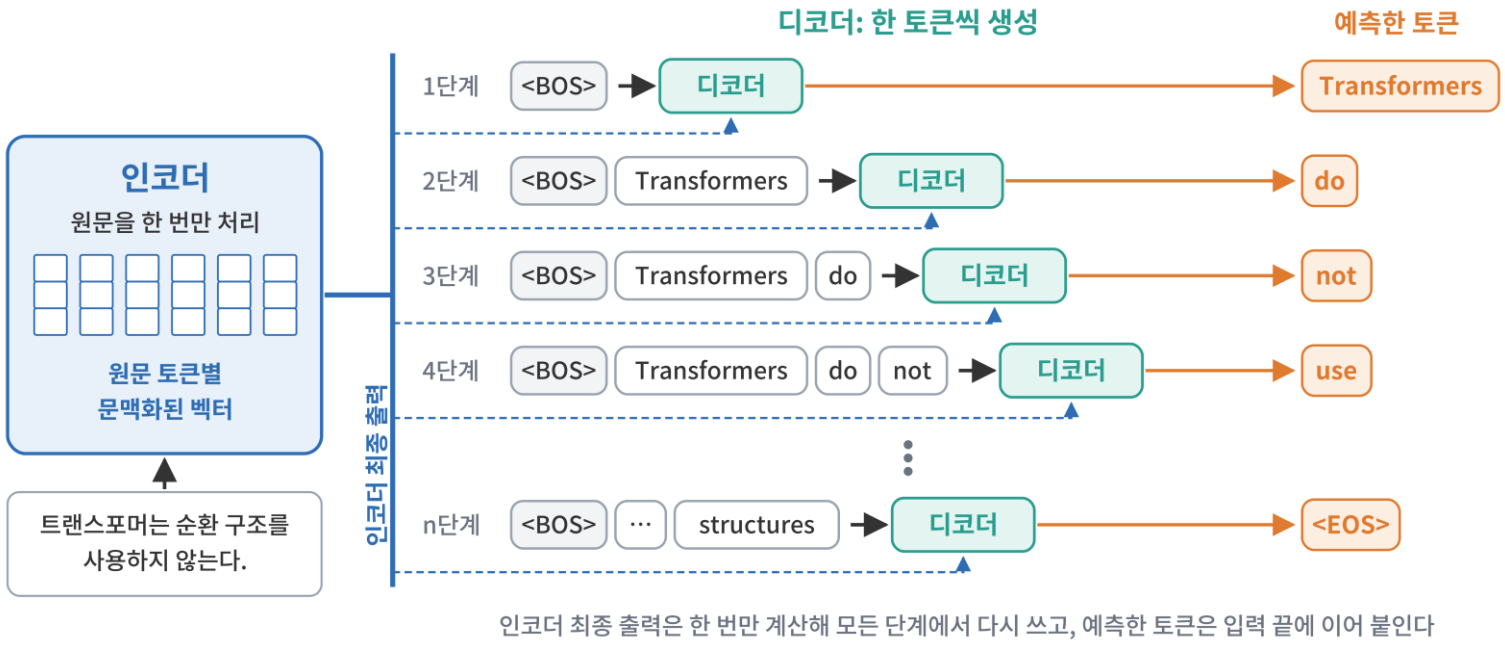


그림. 인코더와 디코더의 관계

- 인코더는 원문을 한 번만 처리해 원문 토큰별 문맥화된 벡터를 만듦
- 디코더는 단계마다 예측한 토큰을 입력 끝에 이어 붙여 <EOS>까지 반복

마스크드 멀티 헤드 셀프 어텐션

- 디코더도 셀프 어텐션으로 지금까지 만든 토큰들 사이의 관계를 계산
- 학습할 때는 정답 번역문 전체를 한꺼번에 넣음
  - 그대로 두면 "not" 다음을 예측할 때 정답 "use"를 미리 봄
  - 답안지를 보며 시험을 치르는 것과 같음
- 마스크(mask): 뒤쪽 토큰을 가려 어텐션 가중치를 0으로 만듦
  - 인과 마스크(causal mask): 앞의 일(원인)만 보고 뒤의 일(결과)을 예측
- 마스크드 멀티 헤드 셀프 어텐션(masked multi-head self-attention)
  - 계산 방법은 인코더의 멀티 헤드 어텐션과 같음

마스크드 셀프 어텐션의 참고 범위

- 토큰마다 참고 범위가 한 칸씩 넓어짐
- 어텐션 맵은 계단 모양
  - 오른쪽 위 절반이 비어 있음
- 정답 전체를 넣어도 앞쪽 정보만으로 다음 토큰을 예측하도록 학습

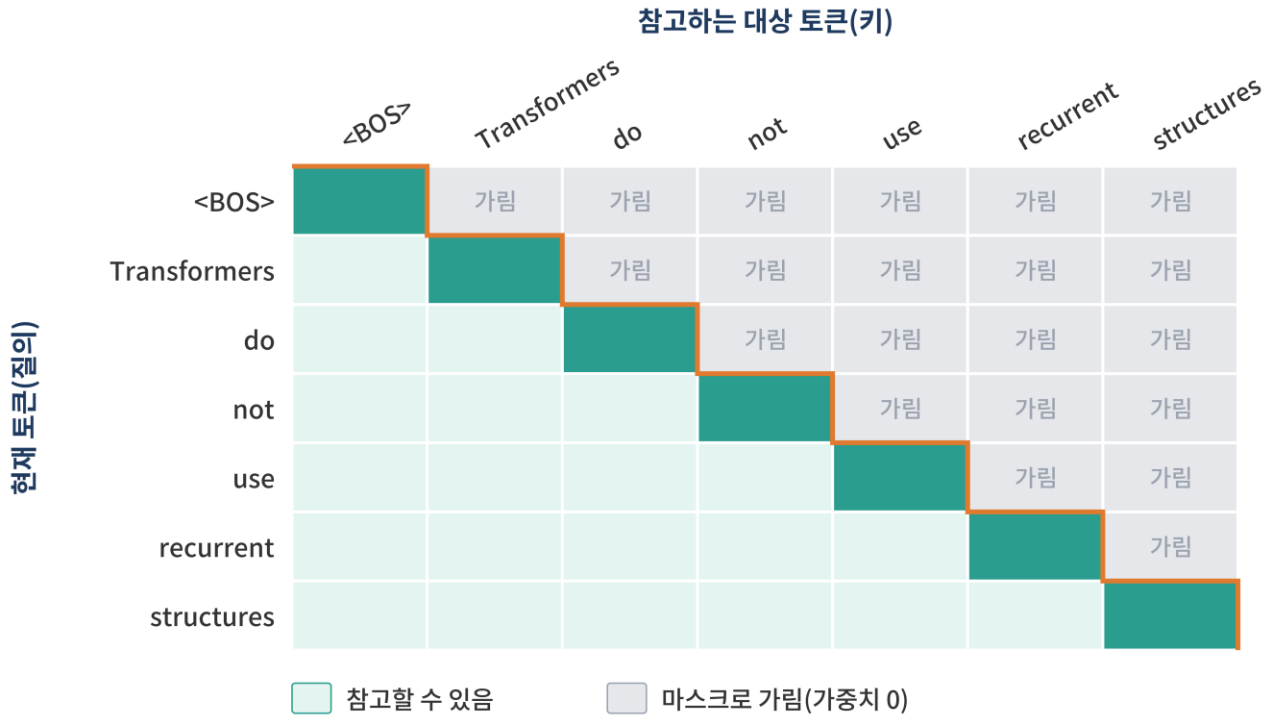


그림. 마스크드 셀프 어텐션에서 토큰별 참고 범위

[IT Lab]

마스크드 셀프 어텐션 직접 확인: Transformer Explainer

- GPT-2(small): 디코더 블록만 12개(768차원, 블록마다 헤드 12개)
  - 이 도구의 셀프 어텐션은 모두 마스크가 적용된 것
- "Transformers do not use"까지 입력하고 Enter

① "Multi-head Self Attention" 패널

- 어텐션 가중치를 점으로 표시한 격자
- 행마다 점이 하나씩 늘어나는 계단 모양 확인

- 점 위에 마우스 → 가중치 값(그림의 0.41)

② 블록, ③ 헤드를 바꿔도 계단 모양은 그대로

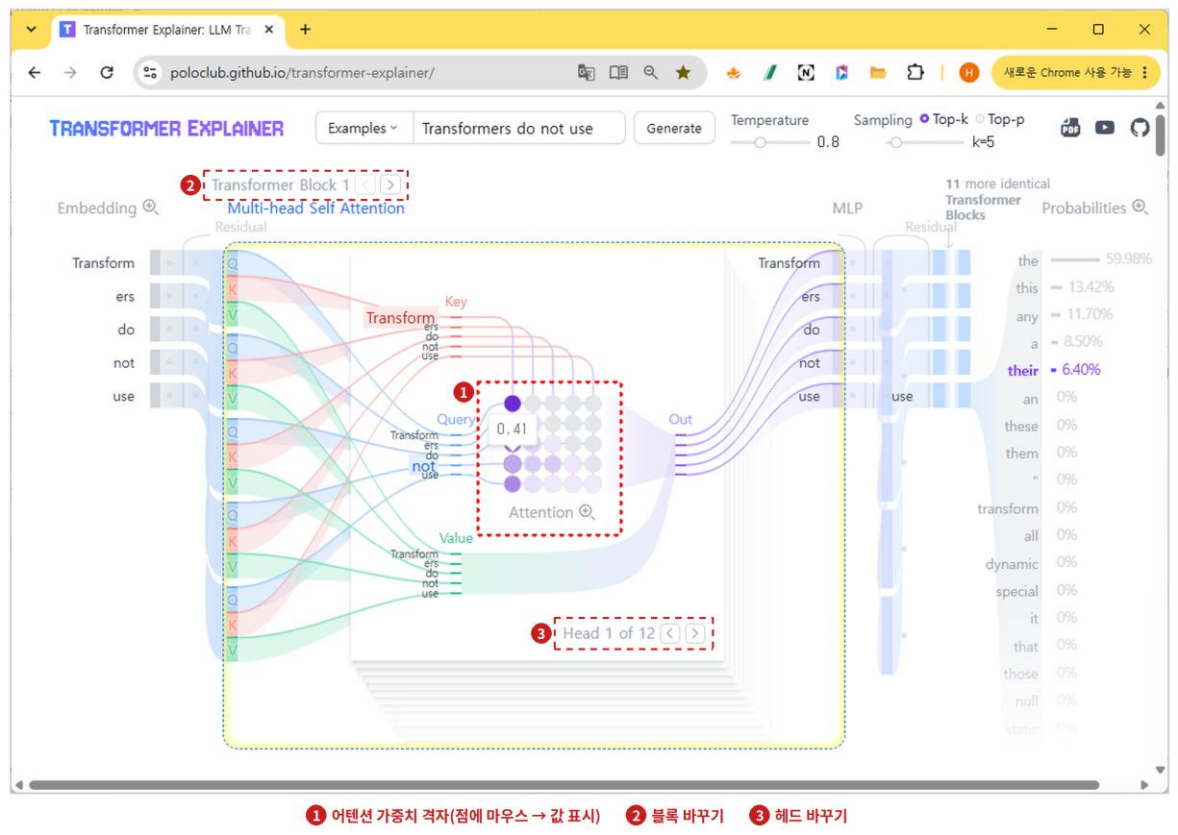


그림. Transformer Explainer의 마스크드 셀프 어텐션 시각화 화면

크로스 어텐션

- 마스크드 셀프 어텐션만으로는 원문 정보를 참고할 수 없음
- 크로스 어텐션(cross-attention): 출력 토큰마다 원문의 어느 부분에 주목할지 판단
  - 다른 문장(원문)을 참고 → '크로스', 인코더-디코더 어텐션이라고도 함
  - RNN 번역 모델의 어텐션이 하던 역할
- 계산 방법은 같지만 질의·키·값의 출처가 다름
  - 질의(Q): 마스크드 셀프 어텐션과 잔차 연결·층 정규화를 마친 디코더 벡터
  - 키(K)·값(V): 인코더의 최종 출력(원문 토큰별 문맥화된 벡터)
- '지금까지 만든 번역문'으로 질문해 관련 깊은 원문 정보를 가져옴

크로스 어텐션에 의한 원문-출력 토큰 정렬

원문 토큰(인코더 최종 출력 → 키·값)

출력 토큰(디코더 → 질의)	트랜스포머	는	순환	구조	를	사용	하지	않는다
Transformers	0.82	0.10	0.02	0.01	0.01	0.02	0.01	0.01
do	0.10	0.02	0.02	0.02	0.02	0.22	0.38	0.22
not	0.03	0.01	0.02	0.02	0.01	0.05	0.36	0.50
use	0.03	0.01	0.03	0.04	0.02	0.78	0.05	0.04
recurrent	0.02	0.01	0.80	0.12	0.01	0.02	0.01	0.01
structures	0.02	0.01	0.14	0.76	0.04	0.01	0.01	0.01

"recurrent"를 만들 때 원문의 "순환"에 가장 크게 주목(0.80) · 각 행의 합 = 1 · 값은 예시

그림. 크로스 어텐션에 의한 원문-출력 토큰 정렬

- "recurrent"를 만들 때 원문의 "순환"에 가장 높은 가중치(0.80)
- 어순이 달라도 위치와 관계없이 필요한 원문 토큰을 찾아 참고(각 행의 합 = 1, 예시 값)

디코더 블록

- 세 단계: 마스크드 멀티 헤드 셀프 어텐션 → 크로스 어텐션 → 피드포워드 신경망
  - 각 단계 뒤에 잔차 연결과 층 정규화, 모두 단계를 건너뛰는 더하기 경로
- 원 논문: 디코더 블록 6개
  - 인코더 최종 출력이 6개 블록의 크로스 어텐션 모두에 전달되어 키·값 생성
- 들어온 토큰 수만큼 벡터를 내보냄
  - 마스크 때문에 각 벡터에는 그 위치까지의 문맥만 담김
- 디코더 전용 모델(이후의 거대언어모델)
  - 원문이 없어 크로스 어텐션이 빠지고 두 단계만 남음

디코더 블록의 구조

- 입력: <BOS>와 지금까지 만든 토큰
- 마스크드 멀티 헤드 셀프 어텐션: 자신과 앞쪽 토큰만 참고
- 크로스 어텐션: 질의는 디코더, 키·값은 인코더
- 피드포워드 신경망(토큰별)
- 각 단계 뒤 잔차 연결과 층 정규화

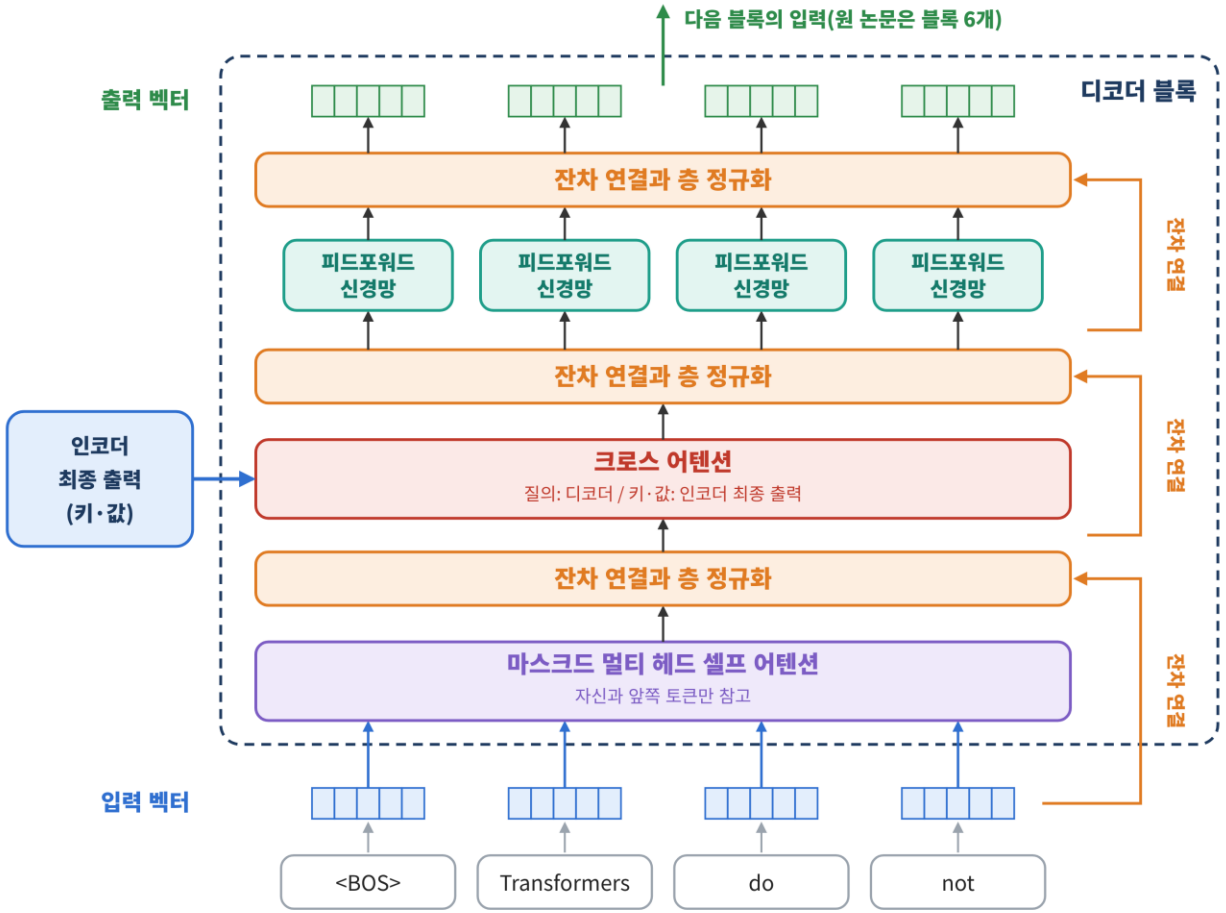


그림. 디코더 블록의 구조

다음 토큰 예측과 출력 확률

- 마지막 블록을 통과한 벡터는 아직 512개의 숫자 → 두 단계로 다음 토큰으로 바꿈
- 마지막 위치의 벡터만 사용: 마스크 때문에 모든 토큰의 정보를 담은 것은 이것뿐

① 로짓(logit): 어휘 사전의 모든 토큰에 매긴 '다음 토큰 적합도' 점수

  - 원 논문 약 37,000개, 확률 이전의 점수라 음수 가능·크기 제한 없음

- 선형 계층(linear layer): 512차원 벡터에 가중치 행렬을 곱해 개수를 바꿈
  - 원 논문은 입력 단계의 임베딩 행렬을 그대로 다시 씀
  - 출력 벡터가 어떤 토큰의 임베딩과 닮을수록 그 로짓이 커짐
- 학습하면서 문맥상 알맞은 토큰의 로짓이 크게 계산되도록 조정됨

선형 계층과 소프트맥스를 통한 다음 토큰 선택

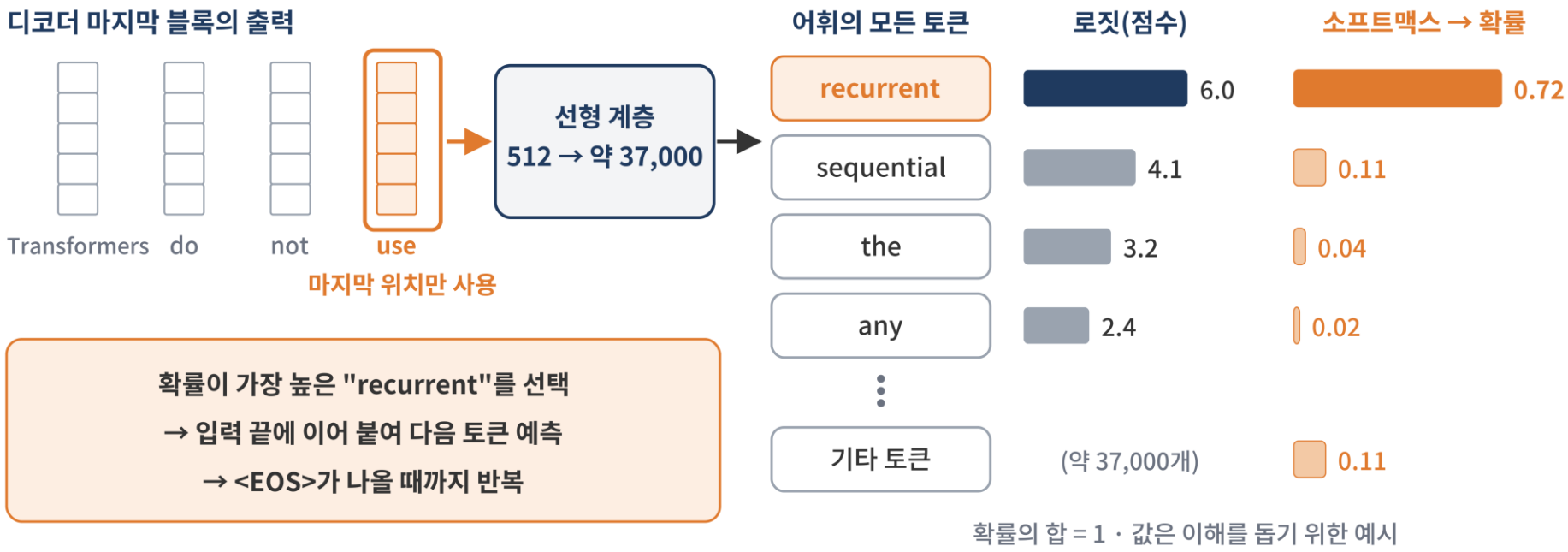


그림. 선형 계층과 소프트맥스를 통한 다음 토큰 선택

② 로짓에 소프트맥스를 적용해 합이 1인 확률로(어텐션 가중치와 같은 계산)

■ "Transformers do not use" → 로짓 recurrent 6.0, sequential 4.1, the 3.2

- 확률 recurrent 약 0.72, sequential 약 0.11, the 약 0.04(예시 값)

다음 토큰을 고르는 방법

- 탐욕적 디코딩(greedy decoding): 확률이 가장 높은 토큰만 고름
  - 같은 입력에 항상 같은 문장 → 표현이 단조로워지기 쉬움
- 샘플링(sampling): 확률에 비례해 무작위로 뽑음(챗GPT 등)
  - 같은 질문에도 답이 조금씩 달라지는 이유
- 온도(temperature): 소프트맥스 전에 로짓을 나누는 값
  - 낮추면 예측 가능한 문장, 높이면 다양한 표현, 지나치면 어색해짐
- 후보 범위: Top-k(상위 k개만), Top-p(누적 확률이 p에 이를 때까지)
  - 온도는 분포의 모양을, Top-k·Top-p는 후보 범위를 정함

[IT Lab]

온도·Top-k·Top-p로 다음 토큰 확률 바꿔보기: Transformer Explainer

- 화면 위쪽: 온도(Temperature), 샘플링 방식(Top-k, Top-p) 메뉴
- "Transformers do not use" 입력
- 온도와 k, p 값을 하나씩 바꾸고 "Generate" 클릭
- 오른쪽 "Probabilities" 열 확인
  - 확률과 선택되는 토큰의 변화
  - 온도를 낮추면 소수의 토큰에 몰림
  - 온도를 높이면 여러 토큰에 고르게 퍼짐

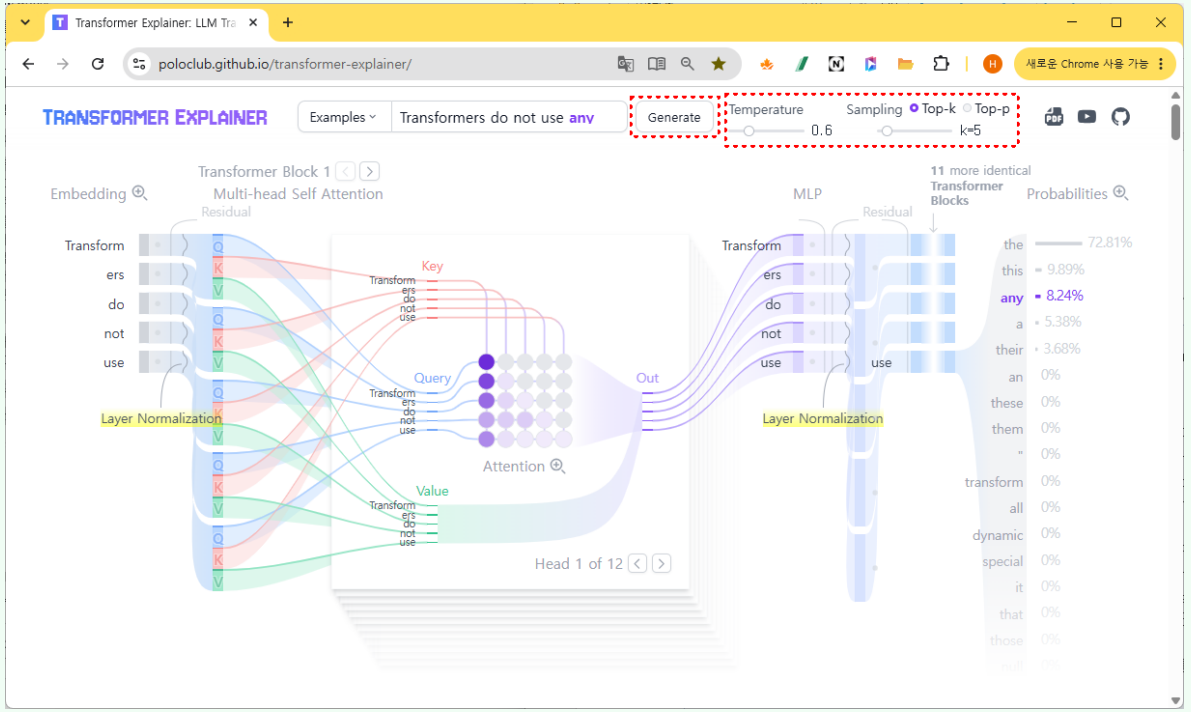


그림. Transformer Explainer의 온도·Top-k·Top-p 조정 화면

트랜스포머의 전체 흐름 정리

- 예: "트랜스포머는 순환 구조를 사용하지 않는다."를 영어로 번역
- 인코더: 토큰화 → 토큰 임베딩 → 위치 인코딩 → 인코더 블록 6개
  - 원문 전체의 문맥이 반영된 토큰별 벡터
- 디코더: <BOS>와 지금까지 만든 토큰 → 디코더 블록 6개
  - 각 블록의 크로스 어텐션은 인코더 최종 출력에서 키·값을 만듦
- 마지막 위치의 벡터 → 선형 계층 → 소프트맥스 → 다음 토큰 확률
- 인코더는 원문을 한 번만, 디코더는 <EOS>까지 토큰을 하나씩 반복 생성

트랜스포머의 전체 구조

- 왼쪽 인코더: 원문을 한 번만 처리, 블록 ×6
- 오른쪽 디코더: 블록 ×6, 크로스 어텐션으로 키·값을 받음
- 마지막 벡터 → 선형 계층·소프트맥스 → 다음 토큰
- 다음 토큰을 입력 끝에 이어 붙여 <EOS>까지 반복

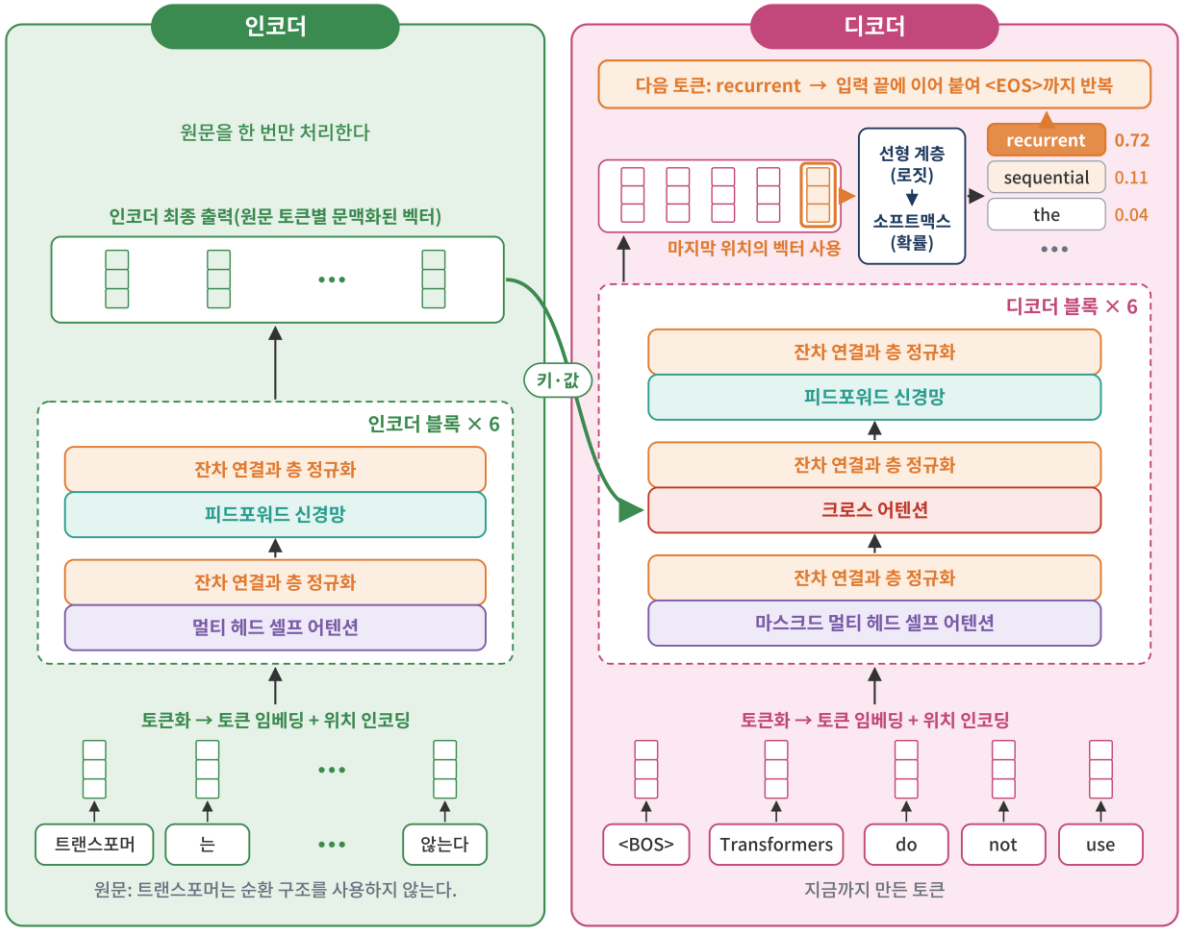


그림. 트랜스포머의 전체 구조

트랜스포머의 학습과 추론

- 학습: 원문과 정답 번역문의 쌍을 대량으로 준비
  - 원문은 인코더에 넣어 한 번에 처리
  - 디코더에는 <BOS>와 정답 번역문 전체를 한꺼번에 넣음
  - 마스크 덕분에 앞쪽만 참고하면서 모든 위치를 동시에 계산
- 티처 포싱(teacher forcing): 디코더 입력으로 항상 정답 토큰을 넣음
  - "do" 다음을 "use"로 잘못 예측해도 다음 위치에는 정답 "not"
  - 앞의 실수가 뒤로 이어지지 않아 빠르고 안정적

트랜스포머의 학습과 추론 (2)

- 추론(inference): 학습을 마친 모델로 실제 문장을 만드는 단계
  - 정답을 알 수 없어 토큰을 하나씩 생성
  - 원문은 인코더에서 한 번만 처리, 디코더는 <BOS>부터 <EOS>까지 반복
- 거대언어모델도 같음
  - 사전학습(pre-training) = 학습, 질문에 답을 만드는 과정 = 추론
- 학습은 문장 전체를 병렬로, 추론은 토큰을 하나씩 순서대로
  - RNN보다 학습은 훨씬 빠르지만, 추론은 문장이 길수록 오래 걸림

학습(티처 포싱)과 추론(순차 생성) 비교

- 학습: 정답 문장을 한꺼번에 입력해 모든 위치를 동시에 계산
  - "not" 대신 "use"로 틀려도 다음 입력은 정답 "not"
- 추론: 앞에서 생성한 토큰을 이어 붙여 하나씩 생성
  - <EOS>까지 반복, 문장이 길수록 오래 걸림

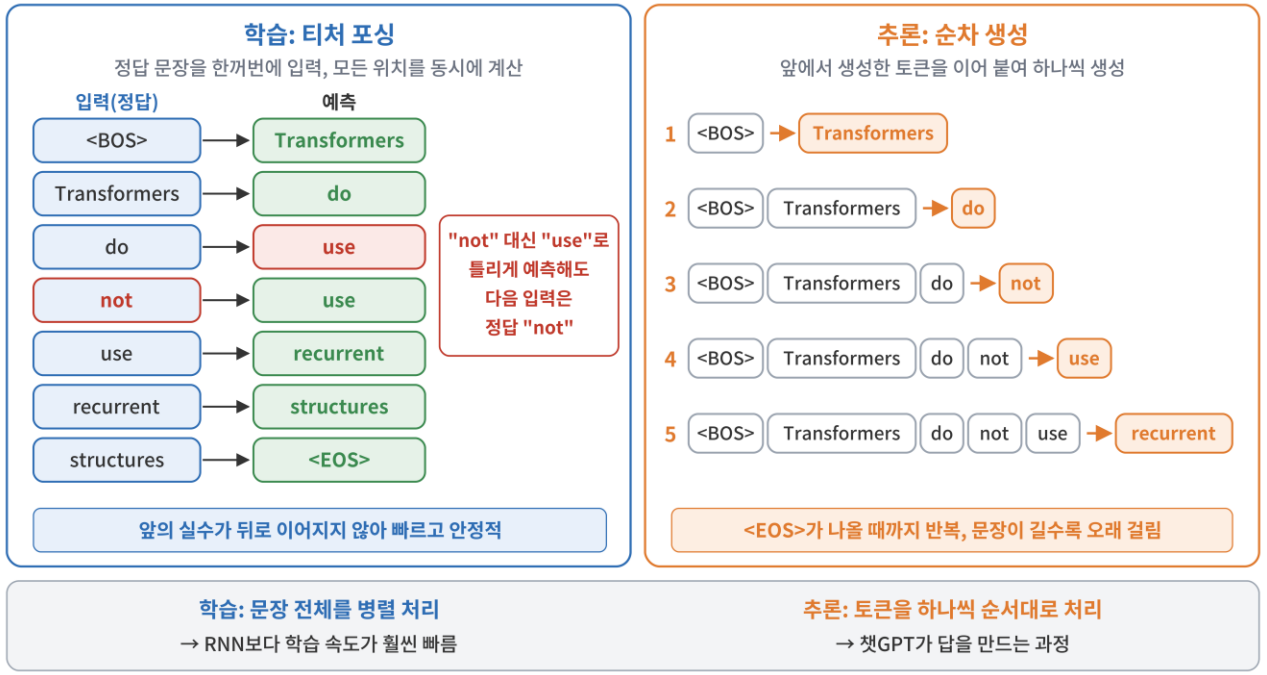


그림. 트랜스포머의 학습(티처 포싱)과 추론(순차 생성) 비교

손실과 역전파를 이용한 학습

- 이전 단원의 신경망과 같은 방식으로 학습
- 손실(loss): 순전파로 계산한 예측이 정답과 얼마나 다른지
  - 교차 엔트로피(cross-entropy): 정답 토큰에 준 확률로 손실을 정함
  - 정답 확률 0.72 → 손실 약 0.33, 0.01 → 약 4.61
- 역전파(backpropagation): 손실을 줄이는 방향(기울기)을 거꾸로 계산해 매개변수 수정
  - 기울기가 크로스 어텐션의 키·값을 통해 인코더와 임베딩 행렬까지 전달
- 원 논문 기본 모델: 약 6,500만 개 매개변수 수정
  - 어텐션 가중치와 위치 인코딩은 학습 대상이 아님

손실 계산과 역전파를 이용한 학습

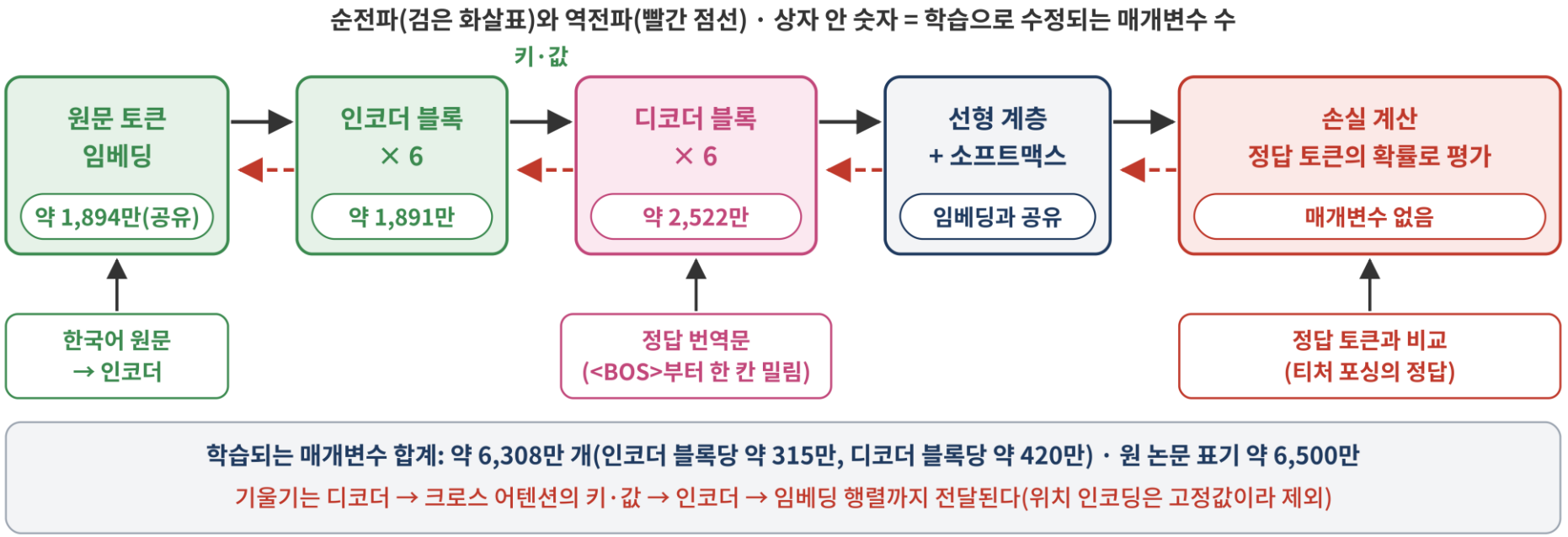


그림. 손실 계산과 역전파를 이용한 트랜스포머의 학습

- 검은 화살표 = 순전파, 빨간 점선 = 역전파, 상자 안 숫자 = 학습으로 수정되는 매개변수 수
- 합계 약 6,308만 개(인코더 블록당 약 315만, 디코더 블록당 약 420만), 원 논문 표기 약 6,500만

트랜스포머 적용 사례

- 원 논문 이후: 인코더와 디코더를 함께 쓰거나 한쪽만 남겨 쓰는 방향으로 발전
- BERT(2018, 구글): 인코더만 쌓음
  - 마스크드 언어 모델링(MLM): 일부 단어를 가리고 앞뒤 문맥으로 맞힘
  - 여기서 '마스크'는 단어를 가린다는 뜻, 디코더의 마스크와 쓰임이 다름
  - 문장을 생성하지 못하지만 감성 분석처럼 뜻을 이해하는 작업에 쓰임
- 미세조정(fine-tuning): 사전학습 모델을 해당 작업 데이터로 조금 더 학습

트랜스포머 적용 사례 (2)

- GPT-2(2019, 오픈AI): 디코더 블록만 쌓아 다음 토큰을 예측
  - 가장 작은 GPT-2(small)는 디코더 블록 12개
- GPT-3(2020): 96개 블록, 1,750억 개 매개변수
  - 미세조정 없이 몇 개의 예시만으로 새로운 작업 수행
  - 이후 단원의 프롬프트 엔지니어링의 토대
- T5(Text-to-Text Transfer Transformer, 2019, 구글): 인코더-디코더 구조
  - 번역·요약·질의응답을 "텍스트 입력 → 텍스트 출력"으로 통일
- 오늘날 거대언어모델: 대부분 디코더 전용 구조
  - 모델이 커질수록 임베딩 차원, 블록 수, 어휘 수가 함께 늘어남

트랜스포머 적용 사례 비교

모델	공개 연도	구조	대표 활용
원 논문 트랜스포머 / T5	2017 / 2019	인코더 + 디코더	번역, 요약, 질의응답
BERT	2018	인코더 전용	문장 이해·분류
GPT-2 / GPT-3	2019 / 2020	디코더 전용	문장 생성

표. 트랜스포머 적용 사례 비교

7.3 정리

1

디코더

인코더의 문맥화된 벡터를 참고해 <BOS>부터 <EOS>까지 한 토큰씩 생성

2

마스크드 셀프 어텐션

자신과 앞쪽 토큰만 참고, 어텐션 맵은 계단 모양

3

크로스 어텐션

질의는 디코더, 키·값은 인코더 최종 출력에서 만들어 원문을 참고

4

다음 토큰 예측

선형 계층으로 로짓 → 소프트맥스로 확률 → 탐욕적 디코딩 또는 샘플링

5

학습과 추론

학습은 티처 포싱으로 병렬 계산, 추론은 토큰을 하나씩 순차 생성

6

적용 사례

인코더+디코더(T5), 인코더 전용(BERT), 디코더 전용(GPT)